FIRST ASSIGNMENT REPORT

Hodgkin-Huxley Neuron Model



Name	: Jeremia Christ Immanuel Manalu
NRP	: 5023231017
Course	: Biocybernetics
Class	: Q
Lecturer	: Dr. Achmad Arifin, S.T., M.Eng.
Department	: Biomedical Engineering

FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS TECHNOLOGY

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

CHAPTER I. FUNDAMENTAL THEORY

1.1 The Hodgkin-Huxley Membrane and Its Equivalent Circuit

Hodgkin and Huxley described the excitable membrane of the squid giant axon as an electrical circuit in which the lipid bilayer acts as a capacitor and every ionic pathway acts as a conductance in series with a battery that represents the Nernst potential of that ion [1]. The circuit used in this assignment follows the week-9 lecture material [2] and is shown in Figure 1.1. The capacitive current J_C charges the membrane capacitance C_m, while the sodium, potassium and leakage currents J_{Na}, J_K and J_L flow through the conductances g_{Na}, g_K and g_L. A stimulus current density J_{in} is injected across the membrane. Because the patch is space clamped, the voltage is uniform over the membrane and Kirchhoff's current law applied to the single node of the circuit gives the current balance of Equation (1.1), from which the voltage equation (1.2) follows directly.

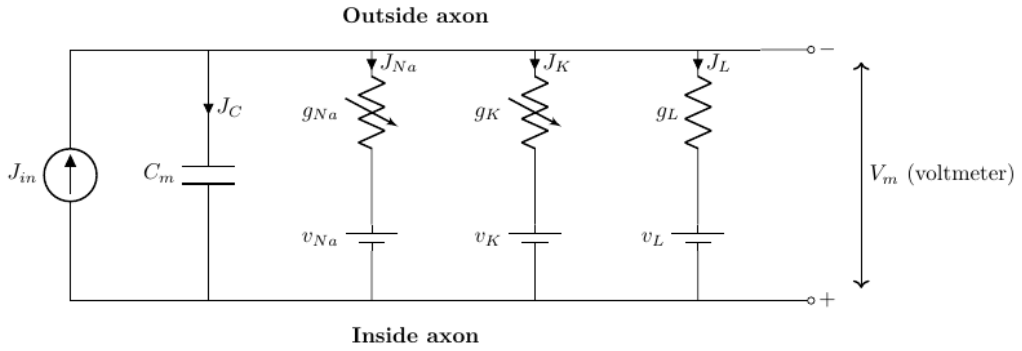


Figure 1.1. *Equivalent circuit of a space-clamped membrane patch in the lecturer's notation (rendered by the author after [1], [2])*

$$J_{in} = C_m \frac{dv}{dt} + J_{Na} + J_K + J_L \qquad (1.1)$$

$$\frac{dv}{dt} = \frac{1}{C_m}[J_{in} - (J_{Na} + J_K + J_L)] \qquad (1.2)$$

The ionic currents obey Ohm's law with a driving force equal to the difference between the membrane voltage and the reversal potential of each ion. Sodium and potassium conductances are not constant, Hodgkin and Huxley expressed them as a maximal conductance multiplied by powers of dimensionless gating variables m, h and n, whereas the leakage conductance is constant [1]:

$$J_{Na} = \bar{g}_{Na}\, m^3 h\, (v - v_{Na}), \qquad J_K = \bar{g}_K\, n^4\, (v - v_K), \qquad J_L = \bar{g}_L\, (v - v_L) \qquad (1.3)$$

A point that must be handled carefully is the sign convention. The original paper and the lecture material [1], [2] define v as the displacement of the membrane potential from rest with depolarization taken as negative, so that $v_{Na} = -115$ mV, $v_K = +12$ mV and $v_L = -10.613$ mV, and an inward (cathodal) stimulus is a negative J_{in}. Modern textbooks use the opposite

sign and absolute potentials. The two descriptions are related by a reflection and a shift, and the program reports the physiological membrane voltage through the alignment of Equation (1.4) with a resting potential of -70 mV. With this alignment the resting state $v = 0$ appears as $V_m = -70$ mV and an action potential appears as a positive excursion that peaks near $+35$ mV. The constants used throughout the work are those of the 1952 paper: $\bar{g}_{Na} = 120$, $\bar{g}_K = 36$ and $\bar{g}_L = 0.3$ mS/cm², $C_m = 1$ µF/cm², at the reference temperature of 6.3 °C.

$$V_m = -(v + 70) \text{ mV} \qquad (1.4)$$

1.2 Gating Kinetics and Rate Constants

Each gating variable represents the fraction of independent molecular gates that are in the permissive state and obeys first-order kinetics with voltage-dependent opening and closing rates $\alpha_x(v)$ and $\beta_x(v)$ [1]. Equation (1.5) can be written equivalently as a relaxation toward the steady state x_∞ with the time constant τ_x of Equation (1.6), which is the form used to interpret the phase plots in Chapter II.

$$\frac{dx}{dt} = \alpha_x(1 - x) - \beta_x x = -x(\alpha_x + \beta_x) + \alpha_x, \qquad x \in \{m, h, n\} \qquad (1.5)$$

$$x_\infty = \frac{\alpha_x}{\alpha_x + \beta_x}, \qquad \tau_x = \frac{1}{\alpha_x + \beta_x} \qquad (1.6)$$

The empirical rate functions fitted by Hodgkin and Huxley to their voltage-clamp data are listed in Equation (1.7) to (1.9) in the lecturer's sign convention, with v in mV and the rates in ms⁻¹ [1], [2]. Two of them, α_m and α_n, are of the form $a(v + b)/\left(e^{c(v+b)} - 1\right)$ and become an indeterminate $0/0$ at $v = -25$ mV and $v = -10$ mV respectively. These are removable singularities whose limits follow from L'Hôpital's rule, $\alpha_m(-25) = 1$ and $\alpha_n(-10) = 0.1$ ms⁻¹. The program evaluates both through the function $x/(e^x - 1)$ of Equation (1.10), which is replaced by its second-order Taylor series when $|x| < 10^{-4}$, instead of adding an arbitrary small number to the denominator.

$$\alpha_m = \frac{0.1\,(v + 25)}{e^{0.1v+2.5} - 1}, \qquad \beta_m = 4\,e^{v/18} \qquad (1.7)$$

$$\alpha_h = 0.07\,e^{v/20}, \qquad \beta_h = \frac{1}{e^{0.1v+3} + 1} \qquad (1.8)$$

$$\alpha_n = \frac{0.01\,(v + 10)}{e^{0.1v+1} - 1}, \qquad \beta_n = 0.125\,e^{v/80} \qquad (1.9)$$

$$\frac{x}{e^x - 1} \approx 1 - \frac{x}{2} + \frac{x^2}{12} \quad \text{for } |x| < 10^{-4} \qquad (1.10)$$

Temperature changes all rate constants by the same factor, written as $\phi = 3^{(T-6.3)/10}$ with T in °C, which corresponds to a Q_{10} of 3 [1]. Figure 1.2 shows the resulting steady states and time constants against the aligned membrane voltage. The sodium activation m is fast (its time constant is below 0.5 ms everywhere), whereas the sodium inactivation h and the

potassium activation n are slower by roughly an order of magnitude. This separation of time scales is the mechanism of the action potential: on depolarization m follows the voltage almost instantly and opens the sodium conductance, which depolarizes the membrane further, while the delayed closure of h and opening of n terminate the spike and produce the after-hyperpolarization and the refractory period.

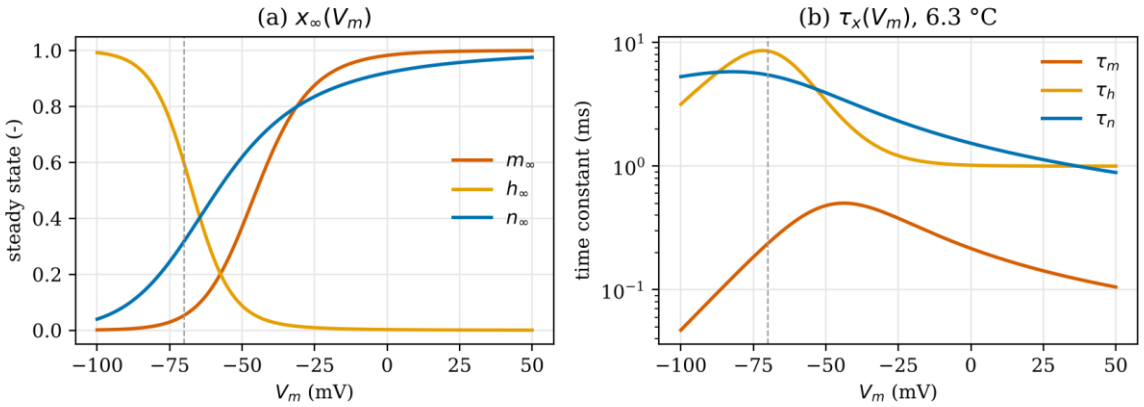


Figure 1.2. *(a) Steady-state values and (b) time constants of the gating variables computed from Equation (1.6) to (1.9) at 6.3 °C, the dashed line marks the resting potential (plotted by the author from the rate functions of [1])*

1.3 Fourth-Order Runge-Kutta Integration

The space-clamped model is a system of four coupled nonlinear ordinary differential equations for the state vector $\mathbf{Y} = [v, m, h, n]^T$, whose right-hand side $\mathbf{F}(t, \mathbf{Y})$ is formed by Equation (1.2) and three copies of Equation (1.5). The lecture material [3] integrates it with the classical fourth-order Runge-Kutta (RK4) method [4], which evaluates four slope estimates per step and combines them with Simpson-like weights:

$$\mathbf{K}_1 = \mathbf{F}(t_k, \mathbf{Y}_k), \qquad \mathbf{K}_2 = \mathbf{F}(t_k + \Delta t/2\,,\ \mathbf{Y}_k + \Delta t/2\,\mathbf{K}_1) \tag{1.11}$$

$$\mathbf{K}_3 = \mathbf{F}(t_k + \Delta t/2\,,\ \mathbf{Y}_k + \Delta t/2\,\mathbf{K}_2), \qquad \mathbf{K}_4 = \mathbf{F}(t_k + \Delta t,\ \mathbf{Y}_k + \Delta t\,\mathbf{K}_3) \tag{1.12}$$

$$\mathbf{Y}_{k+1} = \mathbf{Y}_k + \frac{\Delta t}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4) \tag{1.13}$$

The local truncation error of RK4 is of order Δt^5 and the global error of order Δt^4 [4]. With the step of 0.01 ms used here (the slide prints “10 sec”, which can only be 10 µs because the lecturer’s code works in milliseconds), the scheme resolves the sodium upstroke, whose time scale is about 0.1 ms, with ample margin. The lecturer’s source code [3] applies RK4 in a slightly different way: the rate constants are frozen at v_k, each gate is advanced by its own scalar RK4 step, and the voltage is then advanced with the new gate values held constant. This is an operator-splitting scheme that is first order in the coupling between voltage and gates. Both variants are implemented in the program so that the results can be compared directly with the lecturer’s program.

1.4 The Cable Equation of Action Potential Propagation

To describe propagation, the axon is divided into segments of length Δx that are connected by the internal (axoplasmic) and external resistances per unit length r_i and r_o, as in the transmission-line model of the lecture [2] shown in Figure 1.3 [5]. Every membrane element is either a passive RC element (c_m, g_m) or, in the active cable, the complete Hodgkin-Huxley circuit of Figure 1.1.

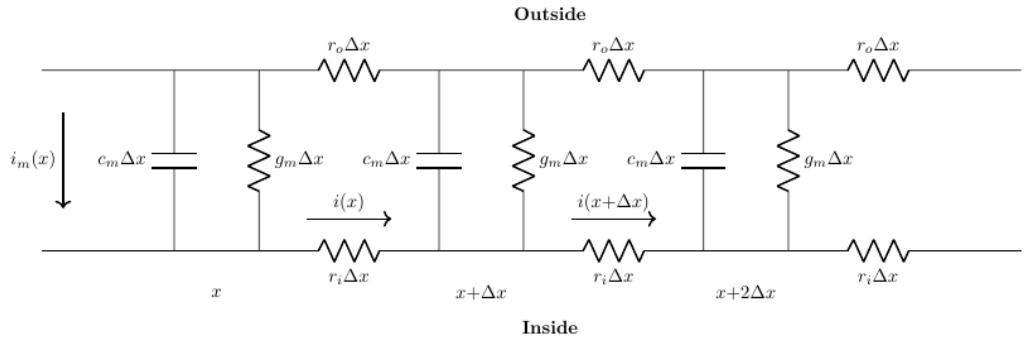


Figure 1.3. *Discretized axon as an RC transmission line (rendered by the author after [2], [5])*

Ohm's law along the axis and Kirchhoff's current law at a node give, in the limit $\Delta x \to 0$, the pair of equations (1.14) and (1.15), where i is the axial current and i_m the membrane current per unit length. Differentiating (1.14) with respect to x and substituting (1.15) eliminates the axial current and yields the cable equation (1.16). Its derivation is repeated step by step as part of Problem 2 in Chapter II.

$$\frac{\partial v}{\partial x} = -(r_o + r_i)i \qquad (1.14)$$

$$\frac{\partial i}{\partial x} = -i_m = -\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (1.15)$$

$$\frac{\partial v}{\partial t} = D\,\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m}, \qquad D = \frac{1}{(r_o + r_i)c_m} \qquad (1.16)$$

For a cylindrical axon of radius a and axoplasm resistivity R_i, the per-length quantities are $r_i = R_i/(\pi a^2)$, $c_m = 2\pi a\, C_m$ and $i_{ion} = 2\pi a\, J_{ion}$, so that $i_{ion}/c_m = J_{ion}/C_m$ [5]. In the passive cable $J_{ion} = g_m v$ and a steady input decays along the axon with the length constant $\lambda = [2\pi a\, g_m(r_o + r_i)]^{-1/2}$, such a cable cannot regenerate a signal. In the active cable $J_{ion} = J_{Na} + J_K + J_L$, so the depolarization that diffuses from an excited region into its neighbour opens the sodium conductance there and regenerates the full action potential. This is why an action potential travels at constant amplitude and constant velocity, and Hodgkin and Huxley showed that the velocity scales with $\sqrt{a}$ and computed 18.8 m/s for their squid axon at 18.5 °C, against 21.2 m/s measured [1]. The lecture slide writes the coefficient of the second derivative as $[(r_o + r_i)C_m]^{-1}$, where C_m is the membrane capacitance per unit length (given there as 1.6×10^{-9} mF/mm), which is the same $D = [(r_o + r_i)c_m]^{-1}$ as in Equation (1.16).

1.5 Finite Difference Approximation and Stability

The cable equation is a parabolic partial differential equation of the diffusion type, the same type as the heat equation $u_t = \alpha u_{xx}$ used in the lecture to introduce its numerical solution [3]. On a grid $x_j = jh$, $t^n = nk$ with $v_j^n \approx v(x_j, t^n)$, the second spatial derivative is replaced by a central difference and the time derivative by a forward difference [4], [6]. The resulting explicit scheme, known as forward-time central-space (FTCS), computes every new value v_j^{n+1} from three known values of the previous time level, as illustrated by the stencil in Figure 1.4.

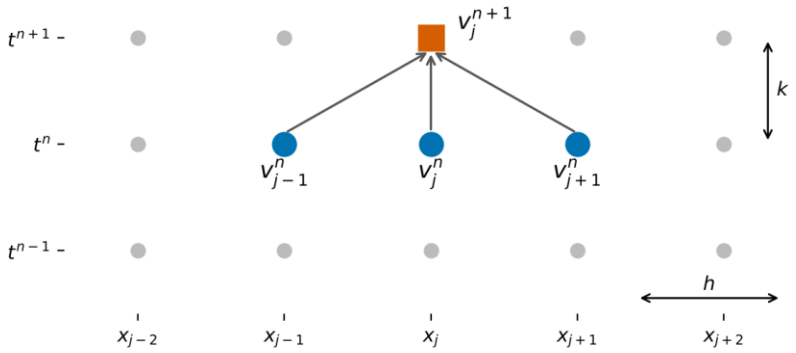


Figure 1.4. *Stencil of the explicit forward-time central-space scheme: the new value (square) is computed from three values of the previous time level (circles)*

The central difference has a truncation error of order h^2 and the forward difference of order k, so the scheme is consistent with an error of $O(k + h^2)$. Because it is explicit, it is only conditionally stable. A von Neumann analysis of the diffusion part shows that the amplification factor of a Fourier mode is $\xi = 1 - 4r\sin^2(\beta h/2)$ with the diffusion number $r = Dk/h^2$, and $|\xi| \leq 1$ for every wave number only if $r \leq 1/2$ [6]. The full derivation of the scheme and of this condition is given as the answer of Problem 2. The program checks the condition before every run and refuses to integrate when it is violated, reporting the largest admissible time step $k_{max} = h^2/(2D)$.

The lecturer's propagation code also contains a Crank-Nicolson variant, which averages the central difference over the old and the new time level. This trapezoidal rule is unconditionally stable and second-order accurate in time for the diffusion term, but it requires a tridiagonal system to be solved at every time step [6]. The program offers both schemes, and the Crank-Nicolson form is derived as an extension of Problem 2 in Section 2.2.3.

1.6 Refractory Period and Repetitive Firing

After an action potential the membrane cannot respond to a second stimulus for a short time and then responds only to a stronger one. During the absolute refractory period the sodium inactivation h is still small and the potassium activation n is still large, so no stimulus, however strong, can make the inward sodium current exceed the outward currents. During the relative

refractory period h recovers and n decays with the time constants of Figure 1.2(b), and the threshold falls back toward its resting value [1]. Operationally, the refractory period is measured with a paired-pulse protocol: a conditioning pulse elicits an action potential and the smallest amplitude of a test pulse that still elicits a second action potential is determined as a function of the interval between the two pulses. A sustained depolarizing current, on the other hand, makes the membrane fire repetitively. After each spike the current recharges the membrane toward threshold, and the larger the current, the faster the threshold is reached once the refractoriness has decayed. The input level therefore programs the inter-pulse interval and hence the firing frequency, up to the point where the membrane remains depolarized, sodium inactivation stays high and firing stops (depolarization block). These two behaviours are the subject of Problem 1 and are quantified in Section 2.3.

CHAPTER II. RESULT AND ANALYSIS

2.1 Problems Statement

1. Based on lecture material in week#9, develop a computer program to simulate Action Potential Generation (APG) using RK4 numerical integration. Describe how to align output from your program to fit the membrane voltage (V_m), show the refractory period, end how the Jin level can program inter-pulse interval.

2. Derive expression of integration of differential partial equation of Action Potential Propagation (APP) based on central difference, and forward difference method.

3. Develop a computer program to simulate the numerical integration method to show the propagation of AP by referring to materials of week#9. Describe how the model can exhibit the propagation.

4. From two scientific papers, write important aspects of HH model in these research works, and the impacts of HH works for advances technology for medical and biology.

A_model_of_the_pacemaking_neuron_of_the_respiratory_central_pattern_generator-1.pdf (28 October 2026, 12:38 PM)

High Speed and Low Digital Resources Im..-1.pdf (28 October 2026, 12:38 PM)

2.2 Answer and Code Explanation

2.2.1 Program Architecture

The program was developed in RAD Studio Delphi 13 Florence (VCL, Object Pascal) and is organized in three units. `HHModel.pas` is the numerical core and contains no user-interface code: it defines the model parameters and state records, the rate constants, both RK4 integrators, the stimulus protocols, the spike detector, the refractory and inter-pulse analyses and the cable solver. `Unit1.pas` with its form file `Unit1.dfm` is the main window, it reads the parameters from editable tables, calls the routines of `HHModel.pas` and draws the results with TeeChart. `EquationViewer.pas` opens every mathematical sheet (equivalent circuit, gating kinetics, RK4, cable equation, finite difference derivation and analysis definitions) in its own window with the caption “Jeremia Christ Immanuel Manalu - 5023231017 - ”, on these sheets, as in this report, the equations that are implemented in the code are highlighted in yellow. Keeping the numerics separate from the form made it possible to compile and regression-test `HHModel.pas` with Free Pascal against an independent Python implementation before it was used in the GUI.

The main window has six tabs: (1) APG with RK4, (2) gating phase plots, (3) refractory period, (4) C-F converter (Jin versus inter-pulse interval), (5) APP with the cable equation and (6) model parameters and equations. Every tab has a Clear button that removes its charts and results while keeping the parameters. Figure 2.1 shows the first tab and Figure 2.2 one of the derivation windows that are opened from the main form. The listings below follow the order

of the problems in Section 2.1 and, within each problem, the order of the source files. Every line carries its line number in the source file. The numerical routines of HHModel.pas are complete, while in the routines of the form the lines that only read parameters, fill memos or repeat a statement for further series are replaced by one comment that names the omitted line range.

All visual elements of the main window are defined at design time in the form file Unit1.dfm, so they can be inspected in the Form Designer of Delphi before the program is compiled. The fourteen TeeChart charts are two-dimensional (View3D = False) and carry their titles, axis titles with units, grid, a legend with check boxes at the bottom and their 48 series (TFastLineSeries, TLineSeries and TPointSeries) in the Okabe-Ito and viridis colours of the program; the parameter tables (TValueListEditor) hold their keys and default values, and tab 6 holds the rendered equations. At run time the code only fills the series with the computed points (for example serVm.AddXY in PlotAPG), sets the titles that depend on a run (recording positions and instants in tab 5) and hides unused series; no series is created or freed while the program runs, which also removes the cause of the access violation recorded as ISS-020. The excerpt below shows the membrane-potential chart of tab 1 in Unit1.dfm (font, grid and position properties are omitted).

```
object chtVm: TChart
  ...
  Legend.Alignment = laBottom
  Legend.CheckBoxes = True
  ...
  Legend.LegendStyle = lsSeries
  ...
  Title.Text.Strings = (
    'Membrane potential and stimulus')
  ...
  BottomAxis.Title.Caption = 't (ms)'
  ...
  LeftAxis.Title.Caption = 'Vm (mV)'
  ...
  RightAxis.Title.Caption = 'Jin (uA/cm2)'
  ...
  View3D = False
  ...
  OnMouseMove = ChartMouseMove
  ...
  OnZoom = ChartZoomed
  object serVm: TFastLineSeries
    SeriesColor = 0
    Title = 'Vm'
    ...
    LinePen.Width = 2
    ...
  end
  object serJin: TFastLineSeries
    SeriesColor = 6316128
    Title = 'Jin (right axis)'
    ...
    LinePen.Width = 2
    VertAxis = aRightAxis
    ...
  end
end
```

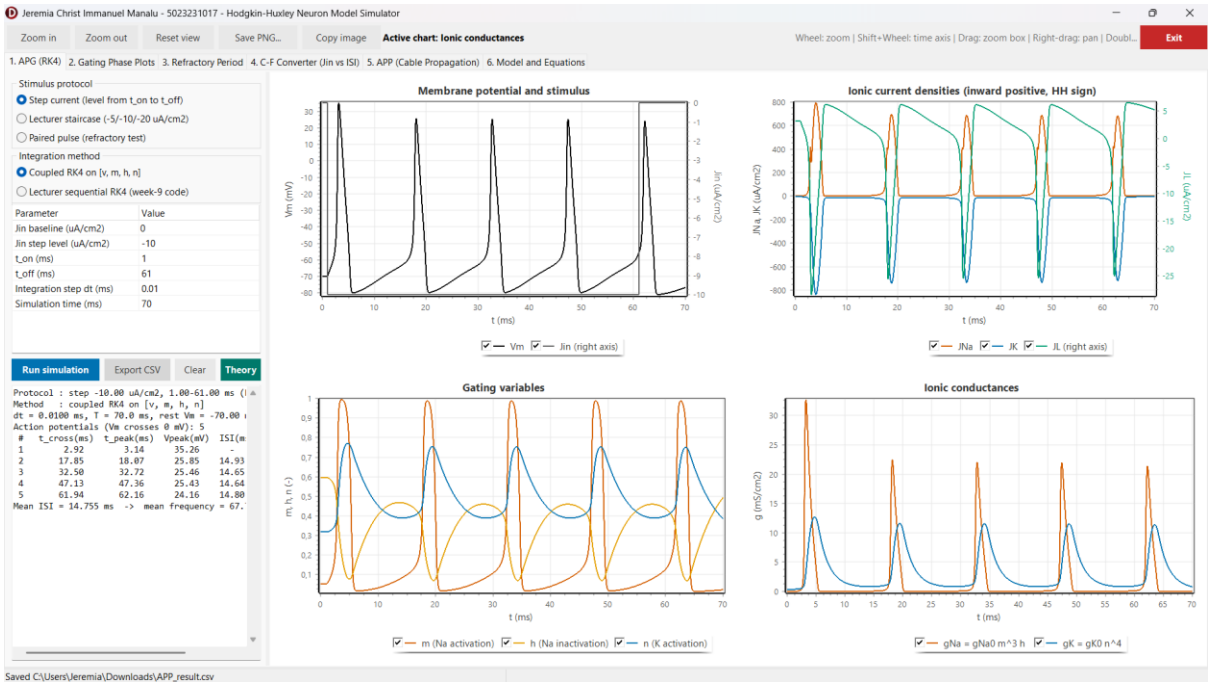


Figure 2.1. *Main window of the simulator, tab "1. APG (RK4)", after a step-current run*

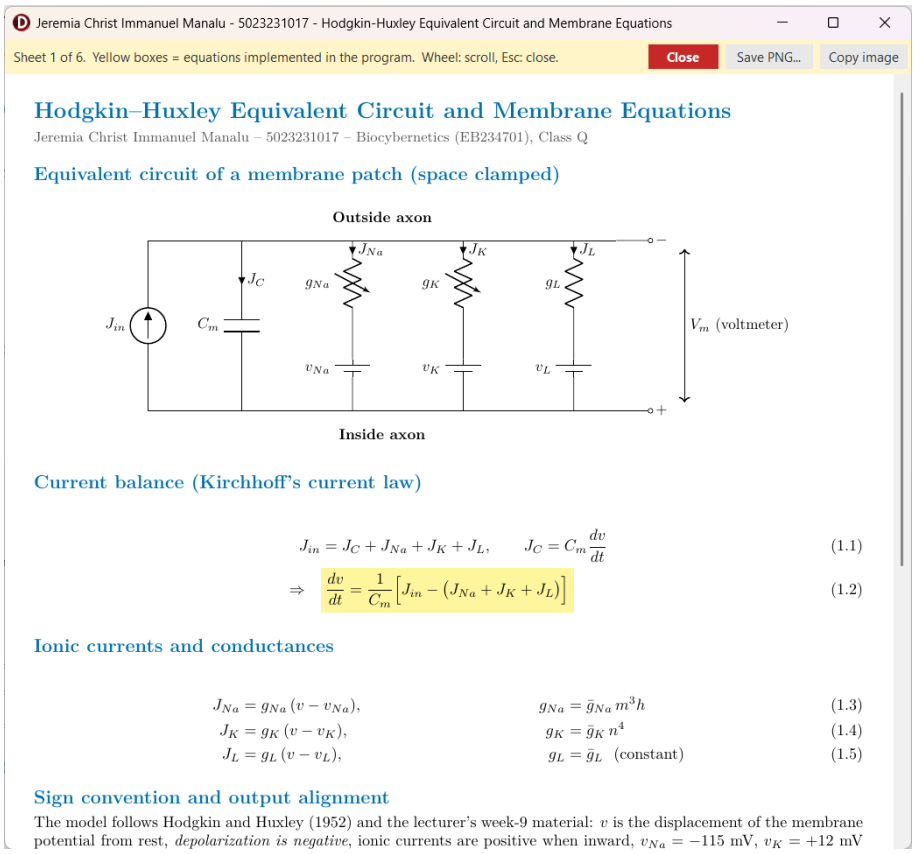


Figure 2.2. *Sheet window opened from the main form (sheet 1, equivalent circuit and membrane equations), yellow boxes mark equations implemented in the program*

2.2.2 Problem 1. Action Potential Generation with RK4

a. Model constants, rate constants and right-hand side

The routines below implement Equation (1.2) to (1.10). `DefaultHHParams` returns the constants of the 1952 paper in the lecturer's sign convention. `TemperatureFactor` computes $\phi = 3^{(T-6.3)/10}$, which multiplies every rate constant. `ExpRatio` evaluates $x/(e^x - 1)$ with the Taylor series near the removable singularity, and `ComputeRates` uses it to write α_m and α_n exactly: since $0.1(v + 25) = 0.1v + 2.5$, the numerator of α_m equals the exponent of its denominator, so $\alpha_m = \text{ExpRatio}(0.1v + 2.5)$, likewise $0.01(v + 10) = 0.1\,(0.1v + 1)$ gives $\alpha_n = 0.1\,\text{ExpRatio}(0.1v + 1)$. `SteadyState` returns x_∞ from Equation (1.6) and is used as the initial condition, so every run starts from the true resting state. `IonicCurrents` implements Equation (1.3), and `RestingConductance` returns the chord conductance $\bar{g}_{Na} m_\infty^3 h_\infty + \bar{g}_K n_\infty^4 + \bar{g}_L$ of the resting membrane, 0.6773 mS/cm², which is used as g_m of the passive cable in Problem 3 so that the passive and active cables have the same resting properties. `Derivatives` assembles $\mathbf{F}(t, \mathbf{Y})$ from Equation (1.2) and (1.5).

```
223  function DefaultHHParams: THHParams;
224  begin
225    Result.GNa0 := 120.0;
226    Result.GK0 := 36.0;
227    Result.GL0 := 0.3;
228    Result.VNa := -115.0;
229    Result.VK := 12.0;
230    Result.VL := -10.613;
231    Result.Cm := 1.0;          // 0.001 mF/cm2 = 1 uF/cm2 (lecturer slide)
232    Result.TempC := 6.3;       // phi = 1, rate equations as published
233  end;
235  function TemperatureFactor(const P: THHParams): Double;
236  begin
237    // Q10 = 3 relative to the 6.3 deg C reference of Hodgkin & Huxley (1952)
238    Result := Power(3.0, (P.TempC - 6.3) / 10.0);
239  end;
241  function ExpRatio(const X: Double): Double;
242  begin
243    // X / (exp(X) - 1) with its removable singularity at X = 0 (limit = 1).
244    // A second-order Taylor expansion is used close to the singularity.
245    if Abs(X) < 1.0E-4 then
246      Result := 1.0 - 0.5 * X + X * X / 12.0
247    else
248      Result := X / (Exp(X) - 1.0);
249  end;
251  function ComputeRates(const V, Phi: Double): THHRates;
252  begin
253    // alpha_m = 0.1(v+25)/(exp(0.1v+2.5)-1) = ExpRatio(0.1v+2.5)
254    Result.Am := Phi * ExpRatio(0.1 * V + 2.5);
255    Result.Bm := Phi * 4.0 * Exp(V / 18.0);
256    Result.Ah := Phi * 0.07 * Exp(V / 20.0);
257    Result.Bh := Phi * 1.0 / (Exp(0.1 * V + 3.0) + 1.0);
258    // alpha_n = 0.01(v+10)/(exp(0.1v+1)-1) = 0.1 * ExpRatio(0.1v+1)
259    Result.An := Phi * 0.1 * ExpRatio(0.1 * V + 1.0);
260    Result.Bn := Phi * 0.125 * Exp(V / 80.0);
261  end;
263  function SteadyState(const V, Phi: Double): THHState;
264  var
265    Rt: THHRates;
266  begin
267    Rt := ComputeRates(V, Phi);
268    Result.V := V;
269    Result.M := Rt.Am / (Rt.Am + Rt.Bm);
270    Result.H := Rt.Ah / (Rt.Ah + Rt.Bh);
271    Result.N := Rt.An / (Rt.An + Rt.Bn);
272  end;
274  procedure IonicCurrents(const P: THHParams; const S: THHState;
275    out JNa, JK, JL: Double);
276  begin
```

```
277   JNa := P.GNa0 * S.M * S.M * S.M * S.H * (S.V - P.VNa);
278   JK := P.GK0 * S.N * S.N * S.N * S.N * (S.V - P.VK);
279   JL := P.GL0 * (S.V - P.VL);
280 end;
282 function RestingConductance(const P: THHParams): Double;
283 var
284   S: THHState;
285 begin
286   // Total (chord) membrane conductance of the resting membrane (mS/cm2);
287   // used as gm of the equivalent passive cable.
288   S := SteadyState(0.0, TemperatureFactor(P));
289   Result := P.GNa0 * S.M * S.M * S.M * S.H + P.GK0 * Sqr(Sqr(S.N)) + P.GL0;
290 end;
292 function Derivatives(const P: THHParams; const Phi: Double;
293   const S: THHState; const Jin: Double): THHState;
294 var
295   Rt: THHRates;
296   JNa, JK, JL: Double;
297 begin
298   Rt := ComputeRates(S.V, Phi);
299   IonicCurrents(P, S, JNa, JK, JL);
300   // Cm dv/dt = Jin - (JNa + JK + JL)
301   Result.V := (Jin - (JNa + JK + JL)) / P.Cm;
302   // dx/dt = alpha_x (1 - x) - beta_x x = -x(alpha_x + beta_x) + alpha_x
303   Result.M := Rt.Am * (1.0 - S.M) - Rt.Bm * S.M;
304   Result.H := Rt.Ah * (1.0 - S.H) - Rt.Bh * S.H;
305   Result.N := Rt.An * (1.0 - S.N) - Rt.Bn * S.N;
306 end;
```

b. Fourth-order Runge-Kutta integration

`StepCoupledRK4` is a direct transcription of Equation (1.11) to (1.13) for the vector $\mathbf{Y} = [v, m, h, n]^T$, `AddScaled` forms the intermediate states $\mathbf{Y}_k + c\,\mathbf{K}$. The stimulus is evaluated at t_k, $t_k + \Delta t/2$ and $t_k + \Delta t$, which is required for fourth-order accuracy when J_{in} changes in time. `StepLecturerRK4` reproduces the week-9 code [3]: the rates are computed once at v_k, `GateRK4` advances each gate with a scalar RK4 step of $dx/dt = \alpha(1-x) - \beta x$, and the voltage is advanced by a scalar RK4 step in which the new gate values are held constant. Writing the gate step for frozen α and β makes clear why this is a splitting scheme: the exact solution of the frozen gate equation is $x_{k+1} = x_\infty + (x_k - x_\infty)e^{-\Delta t/\tau_x}$, which RK4 reproduces to $O(\Delta t^5)$, but the voltage dependence of α and β within the step is neglected, which introduces a first-order coupling error. With $\Delta t = 0.01$ ms the two methods differ by at most 0.01 ms in spike times in all tested cases.

```
308 function AddScaled(const A, B: THHState; const C: Double): THHState; inline;
309 begin
310   // A + C * B
311   Result.V := A.V + C * B.V;
312   Result.M := A.M + C * B.M;
313   Result.H := A.H + C * B.H;
314   Result.N := A.N + C * B.N;
315 end;
317 function StepCoupledRK4(const P: THHParams; const Phi: Double;
318   const S: THHState; const J0, JHalf, J1, Dt: Double): THHState;
319 var
320   K1, K2, K3, K4: THHState;
321 begin
322   // Classical 4th-order Runge-Kutta on Y = [v, m, h, n]:
323   //   K1 = F(t, Y),            K2 = F(t + dt/2, Y + dt/2 K1)
324   //   K3 = F(t + dt/2, Y + dt/2 K2), K4 = F(t + dt, Y + dt K3)
325   //   Y(t+dt) = Y + dt/6 (K1 + 2K2 + 2K3 + K4)
326   K1 := Derivatives(P, Phi, S, J0);
327   K2 := Derivatives(P, Phi, AddScaled(S, K1, 0.5 * Dt), JHalf);
328   K3 := Derivatives(P, Phi, AddScaled(S, K2, 0.5 * Dt), JHalf);
```

```
  K4 := Derivatives(P, Phi, AddScaled(S, K3, Dt), J1);
  Result.V := S.V + Dt / 6.0 * (K1.V + 2.0 * K2.V + 2.0 * K3.V + K4.V);
  Result.M := S.M + Dt / 6.0 * (K1.M + 2.0 * K2.M + 2.0 * K3.M + K4.M);
  Result.H := S.H + Dt / 6.0 * (K1.H + 2.0 * K2.H + 2.0 * K3.H + K4.H);
  Result.N := S.N + Dt / 6.0 * (K1.N + 2.0 * K2.N + 2.0 * K3.N + K4.N);
end;

function GateRK4(const X, A, B, Dt: Double): Double;
var
  K1, K2, K3, K4: Double;
begin
  // Scalar RK4 of dx/dt = A(1-x) - Bx with A, B frozen during the step
  K1 := Dt * (A * (1.0 - X) - B * X);
  K2 := Dt * (A * (1.0 - (X + 0.5 * K1)) - B * (X + 0.5 * K1));
  K3 := Dt * (A * (1.0 - (X + 0.5 * K2)) - B * (X + 0.5 * K2));
  K4 := Dt * (A * (1.0 - (X + K3)) - B * (X + K3));
  Result := X + (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0;
end;

function StepLecturerRK4(const P: THHParams; const Phi: Double;
  const S: THHState; const Jin, Dt: Double): THHState;
var
  Rt: THHRates;
  Tmp: THHState;
  K1, K2, K3, K4: Double;

  function DV(const V: Double): Double;
  var
    JNa, JK, JL: Double;
  begin
    Tmp.V := V;
    IonicCurrents(P, Tmp, JNa, JK, JL);
    Result := (Jin - (JNa + JK + JL)) / P.Cm;
  end;

begin
  // Reproduces the week-9 source code: alpha/beta are evaluated once at
  // v(t); n, m, h are advanced by independent scalar RK4 steps; then v is
  // advanced by a scalar RK4 step with the NEW gate values held constant.
  Rt := ComputeRates(S.V, Phi);
  Result.N := GateRK4(S.N, Rt.An, Rt.Bn, Dt);
  Result.M := GateRK4(S.M, Rt.Am, Rt.Bm, Dt);
  Result.H := GateRK4(S.H, Rt.Ah, Rt.Bh, Dt);
  Tmp := Result;
  K1 := Dt * DV(S.V);
  K2 := Dt * DV(S.V + 0.5 * K1);
  K3 := Dt * DV(S.V + 0.5 * K2);
  K4 := Dt * DV(S.V + K3);
  Result.V := S.V + (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0;
end;
```

c. Alignment of the program output to the membrane voltage

The integrator works with the displacement v of the lecturer's convention, in which rest is $v = 0$ and depolarization is negative. The physiological membrane voltage is obtained by reflecting the sign and adding the resting potential. If V denotes the modern (depolarization-positive) displacement, the two conventions are related by $V = -v$, and the absolute membrane voltage is the resting potential plus this displacement:

$$V_m = V_{rest} + V = V_{rest} - v \qquad (2.1)$$

$$V_m = -70 - v = -(v + 70)\ \text{mV}, \qquad v = 0 \Rightarrow V_m = -70\ \text{mV}, \qquad v = -105 \Rightarrow V_m = +35\ \text{mV} \qquad (2.2)$$

The same reflection explains the stimulus sign, a depolarizing (inward) current must make v more negative, so it enters Equation (1.2) as a negative J_{in}. `AbsoluteVm` implements Equation (2.2), it is the only place where the alignment is made, and every chart, table and CSV

export uses its result, while the reversal potentials remain in the lecturer's convention ($v_{Na} = -115$ mV corresponds to $V_m = +45$ mV and $v_K = +12$ mV to $V_m = -82$ mV).

```
380  function AbsoluteVm(const V: Double): Double;
381  begin
382    // Alignment of the HH displacement v to the absolute membrane voltage
383    // Vm = -(v + 70) = V_REST_ABS - v
384    Result := V_REST_ABS - V;
385  end;
```

d. Stimulus protocols, APG run and spike detection

`StimulusAt` returns $J_{in}(t)$ for the three protocols of tab 1: a step from t_{on} to t_{off}, the lecturer's staircase (-5, -10 and -20 µA/cm² on [0,25), [25,38) and [38,50) ms) and the paired pulse used for refractoriness. `AdvanceState` selects the integrator and `CheckStepArguments` rejects non-positive or excessive step counts. `SimulateAPG` starts from the resting steady state, stores t, J_{in}, v, V_m, the three ionic currents, the gates and the conductances $g_{Na} = \bar{g}_{Na} m^3 h$ and $g_K = \bar{g}_K n^4$ at every step, and feeds V_m to the spike detector. An action potential is defined by an upward crossing of $V_m = 0$ mV, `TSpikeDetector.Feed` records the crossing time, follows the peak until the downward crossing and then stores the spike, and `TSpikeDetector.Finish` also counts a spike that is still above 0 mV when the run ends.

```
389  procedure AppendValue(var A: TDoubleArray; const Value: Double);
390  var
391    L: Integer;
392  begin
393    L := Length(A);
394    SetLength(A, L + 1);
395    A[L] := Value;
396  end;
398  procedure TSpikeDetector.Reset;
399  begin
400    Above := False;
401    PeakV := -1.0E30;
402    PeakT := 0.0;
403    CrossT := 0.0;
404    Count := 0;
405    SetLength(CrossTimes, 0);
406    SetLength(PeakTimes, 0);
407    SetLength(PeakValues, 0);
408  end;
410  procedure TSpikeDetector.Feed(const T, Vm: Double);
411  begin
412    if not Above then
413    begin
414      if Vm >= SPIKE_LEVEL then
415      begin
416        Above := True;
417        CrossT := T;
418        PeakV := Vm;
419        PeakT := T;
420      end;
421    end
422    else
423    begin
424      if Vm > PeakV then
425      begin
426        PeakV := Vm;
427        PeakT := T;
```

```
428       end;
429       if Vm < SPIKE_LEVEL then
430       begin
431         Above := False;
432         AppendValue(CrossTimes, CrossT);
433         AppendValue(PeakTimes, PeakT);
434         AppendValue(PeakValues, PeakV);
435         Inc(Count);
436       end;
437     end;
438   end;
440   procedure TSpikeDetector.Finish;
441   begin
442     // A spike still above the detection level when the run ends is counted
443     if Above then
444     begin
445       AppendValue(CrossTimes, CrossT);
446       AppendValue(PeakTimes, PeakT);
447       AppendValue(PeakValues, PeakV);
448       Inc(Count);
449       Above := False;
450     end;
451   end;
455   function DefaultStimulus: TStimulus;
456   begin
457     Result.Kind := skStep;
458     Result.JBase := 0.0;
459     Result.JLevel := -10.0;
460     Result.TOn := 1.0;
461     Result.TOff := 61.0;
462     Result.PulseWidth := 0.5;
463     Result.Interval := 15.0;
464     Result.JLevel2 := -40.0;
465   end;
467   function StimulusAt(const Stim: TStimulus; const T: Double): Double;
468   begin
469     case Stim.Kind of
470       skStep:
471         if (T >= Stim.TOn) and (T < Stim.TOff) then
472           Result := Stim.JLevel
473         else
474           Result := Stim.JBase;
475       skLecturerStaircase:
476         // identical to the lecturer's week-9 source code
477         if (T >= 0.0) and (T < 25.0) then
478           Result := -5.0
479         else if (T >= 25.0) and (T < 38.0) then
480           Result := -10.0
481         else if (T >= 38.0) and (T < 50.0) then
482           Result := -20.0
483         else
484           Result := 0.0;
485       skPairedPulse:
486         if (T >= Stim.TOn) and (T < Stim.TOn + Stim.PulseWidth) then
487           Result := Stim.JLevel
488         else if (T >= Stim.TOn + Stim.Interval) and
489                 (T < Stim.TOn + Stim.Interval + Stim.PulseWidth) then
490           Result := Stim.JLevel2
491         else
492           Result := Stim.JBase;
493     else
494       Result := 0.0;
495     end;
496   end;
500   function AdvanceState(const P: THHParams; const Phi: Double;
501     const Stim: TStimulus; const S: THHState; const T, Dt: Double;
502     const Method: TIntegrator): THHState;
503   begin
504     if Method = imCoupledRK4 then
505       Result := StepCoupledRK4(P, Phi, S, StimulusAt(Stim, T),
506         StimulusAt(Stim, T + 0.5 * Dt), StimulusAt(Stim, T + Dt), Dt)
507     else
```

```
508       Result := StepLecturerRK4(P, Phi, S, StimulusAt(Stim, T), Dt);
509   end;
511   procedure CheckStepArguments(const Dt, TEnd: Double);
512   begin
513     if (Dt <= 0.0) or IsNan(Dt) then
514       raise EArgumentException.Create('Integration step dt must be positive.');
515     if (TEnd <= 0.0) or IsNan(TEnd) then
516       raise EArgumentException.Create('Simulation time must be positive.');
517     if TEnd / Dt > 5.0E6 then
518       raise EArgumentException.Create(
519         'Too many integration steps (TEnd/dt > 5e6). Increase dt or reduce TEnd.');
520   end;
522   procedure SimulateAPG(const P: THHParams; const Stim: TStimulus;
523     const Dt, TEnd: Double; const Method: TIntegrator; out R: TAPGResult);
524   var
525     Phi, T: Double;
526     S: THHState;
527     I, NSteps: Integer;
528     JNa, JK, JL: Double;
529     Det: TSpikeDetector;
530
531     procedure Store(const Idx: Integer; const Time, Jin: Double);
532     begin
533       IonicCurrents(P, S, JNa, JK, JL);
534       R.T[Idx] := Time;
535       R.V[Idx] := S.V;
536       R.Vm[Idx] := AbsoluteVm(S.V);
537       R.Jin[Idx] := Jin;
538       R.JNa[Idx] := JNa;
539       R.JK[Idx] := JK;
540       R.JL[Idx] := JL;
541       R.M[Idx] := S.M;
542       R.H[Idx] := S.H;
543       R.N[Idx] := S.N;
544       R.GNa[Idx] := P.GNa0 * S.M * S.M * S.M * S.H;
545       R.GK[Idx] := P.GK0 * Sqr(Sqr(S.N));
546     end;
547
548   begin
549     CheckStepArguments(Dt, TEnd);
550     Phi := TemperatureFactor(P);
551     NSteps := Round(TEnd / Dt);
552     R.Count := NSteps + 1;
553     SetLength(R.T, R.Count);
554     SetLength(R.Vm, R.Count);
555     SetLength(R.V, R.Count);
556     SetLength(R.Jin, R.Count);
557     SetLength(R.JNa, R.Count);
558     SetLength(R.JK, R.Count);
559     SetLength(R.JL, R.Count);
560     SetLength(R.M, R.Count);
561     SetLength(R.H, R.Count);
562     SetLength(R.N, R.Count);
563     SetLength(R.GNa, R.Count);
564     SetLength(R.GK, R.Count);
565
566     // Initial condition: membrane at rest, gates at their steady state
567     S := SteadyState(0.0, Phi);
568     Det.Reset;
569     Store(0, 0.0, StimulusAt(Stim, 0.0));
570     Det.Feed(0.0, R.Vm[0]);
571     for I := 1 to NSteps do
572     begin
573       T := (I - 1) * Dt;
574       S := AdvanceState(P, Phi, Stim, S, T, Dt, Method);
575       Store(I, I * Dt, StimulusAt(Stim, I * Dt));
576       Det.Feed(I * Dt, R.Vm[I]);
577     end;
578     Det.Finish;
579     R.SpikeCross := Copy(Det.CrossTimes);
580     R.SpikeTimes := Copy(Det.PeakTimes);
581     R.SpikePeaks := Copy(Det.PeakValues);
582   end;
```

```
584  procedure SimulateSpikesOnly(const P: THHParams; const Stim: TStimulus;
585    const Dt, TEnd: Double; const Method: TIntegrator;
586    var Det: TSpikeDetector);
587  var
588    Phi: Double;
589    S: THHState;
590    I, NSteps: Integer;
591  begin
592    CheckStepArguments(Dt, TEnd);
593    Phi := TemperatureFactor(P);
594    NSteps := Round(TEnd / Dt);
595    S := SteadyState(0.0, Phi);
596    Det.Reset;
597    Det.Feed(0.0, AbsoluteVm(S.V));
598    for I := 1 to NSteps do
599    begin
600      S := AdvanceState(P, Phi, Stim, S, (I - 1) * Dt, Dt, Method);
601      Det.Feed(I * Dt, AbsoluteVm(S.V));
602    end;
603    Det.Finish;
604  end;
```

On the form, `btnRunAPGClick` reads the protocol and the parameters, validates them, runs `SimulateAPG` and calls the plotting and report routines. `PlotAPG` draws V_m with J_{in} on a right axis, the three ionic currents (J_L on its own right axis because it is two orders of magnitude smaller than J_{Na} and J_K), the gates and the conductances, the four time axes are synchronized when zooming. `PlotPhase` draws the trajectories of m, h and n against V_m as dots at a constant time stride over the steady-state curves $x_\infty(V_m)$, together with $\tau_x(V_m)$ on a logarithmic axis, which makes the lag of each gate behind its steady state visible. `ReportAPG` lists the spike times, peaks and inter-spike intervals in the result memo.

```
1390  procedure TForm1.BuildAPGStimulus(out AStim: TStimulus);
1391  begin
1392    // stimulus of tab 1; also used as the source of the APG waveform that
1393    // drives the cable at x = 0 in tab 5
1394    AStim := DefaultStimulus;
1395    case rgProtocol.ItemIndex of
1396      1:
1397        AStim.Kind := skLecturerStaircase;
1398      2:
1399        begin
1400          AStim.Kind := skPairedPulse;
                { lines 1401-1406 omitted: J base, t_on, J1, J2, width, interval }
1407          if AStim.PulseWidth <= 0.0 then
1408            raise EConvertError.Create('Pulse width must be positive.');
1409          if AStim.Interval < AStim.PulseWidth then
1410            raise EConvertError.Create(
1411              'Pulse interval must not be shorter than the pulse width.');
1412        end;
1413    else
1414      AStim.Kind := skStep;
1415      AStim.JBase := ParamValue(vleAPG, K_JBASE);
1416      AStim.JLevel := ParamValue(vleAPG, K_JLEVEL);
1417      AStim.TOn := ParamValue(vleAPG, K_TON);
1418      AStim.TOff := ParamValue(vleAPG, K_TOFF);
1419      if AStim.TOff <= AStim.TOn then
1420        raise EConvertError.Create('t_off must be larger than t_on.');
1421    end;
1422  end;
1424  procedure TForm1.btnRunAPGClick(Sender: TObject);
1425  var
1426    LParams: THHParams;
1427    LStim: TStimulus;
1428    LDt, LTEnd: Double;
1429  begin
1430    try
1431      LParams := ReadModelParams;
```

```
1432      BuildAPGStimulus(LStim);
1433      LDt := ParamValue(vleAPG, K_DT);
1434      LTEnd := ParamValue(vleAPG, K_TEND);
1435
1436      Screen.Cursor := crHourGlass;
1437      try
1438        SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, FAPG);
1439      finally
1440        Screen.Cursor := crDefault;
1441      end;
1442      FAPGValid := True;
1443      PlotAPG;
1444      PlotPhase(LParams);
1445      ReportAPG(LStim, LDt, LTEnd);
1446      SetStatus(Format('APG finished: %d samples, %d action potentials.',
1447        [FAPG.Count, Length(FAPG.SpikeTimes)]));
1448    except
1449      on E: Exception do
1450        ShowError(E.Message);
1451    end;
1452  end;
1454  procedure TForm1.PlotAPG;
1455  var
1456    LIdx, LStride: Integer;
1457  begin
1458    // series of the four APG charts are designed in Unit1.dfm; JL (about
1459    // +/-25 uA/cm2) is on the right axis because it would be invisible next
1460    // to JNa and JK (about +/-800 uA/cm2), Jin is on the right axis of chtVm
1461    ClearChart(chtVm);
1462    ClearChart(chtCurrents);
1463    ClearChart(chtGates);
1464    ClearChart(chtConductance);
1465    // at most about 8000 points per series keeps TeeChart responsive
1466    LStride := Max(1, FAPG.Count div 8000);
1467    LIdx := 0;
1468    while LIdx < FAPG.Count do
1469    begin
1470      serVm.AddXY(FAPG.T[LIdx], FAPG.Vm[LIdx]);
       { lines 1471-1479 omitted: the other nine series in the same way }
1480      Inc(LIdx, LStride);
1481    end;
1482    FinishChart(chtVm);
       { lines 1483-1485 omitted: FinishChart of the other charts }
1486  end;
1488  procedure TForm1.PlotPhase(const AParams: THHParams);
1489  const
1490    C_VM_FROM = -100.0;   // range of the steady-state curves (mV)
1491    C_VM_TO = 50.0;
1492    C_VM_STEP = 0.5;
1493  var
1494    LIdx, LStride, LCount: Integer;
1495    LPhi, LVm, LV: Double;
1496    LRates: THHRates;
1497  begin
       { lines 1498-1506 omitted: comment, ClearChart of the four charts }
1507
1508    LStride := Max(1, FAPG.Count div 4000);
1509    LIdx := 0;
1510    while LIdx < FAPG.Count do
1511    begin
1512      serTrajM.AddXY(FAPG.Vm[LIdx], FAPG.M[LIdx]);
1513      serTrajH.AddXY(FAPG.Vm[LIdx], FAPG.H[LIdx]);
1514      serTrajN.AddXY(FAPG.Vm[LIdx], FAPG.N[LIdx]);
1515      Inc(LIdx, LStride);
1516    end;
1517
1518    LPhi := TemperatureFactor(AParams);
1519    LCount := Round((C_VM_TO - C_VM_FROM) / C_VM_STEP);
1520    for LIdx := 0 to LCount do
1521    begin
1522      LVm := C_VM_FROM + LIdx * C_VM_STEP;
1523      LV := V_REST_ABS - LVm;            // HH displacement: v = -(Vm + 70)
1524      LRates := ComputeRates(LV, LPhi);
```

```
1525      serInfM.AddXY(LVm, LRates.Am / (LRates.Am + LRates.Bm));
          { lines 1526-1530 omitted: h_inf, n_inf, tau_m, tau_h, tau_n alike }
1531    end;
        { lines 1532-1535 omitted: FinishChart of the four charts }
1536  end;
1538  procedure TForm1.ReportAPG(const AStim: TStimulus; const ADt, ATEnd: Double);
1539  var
1540    LLines: TStrings;
1541    LIdx, LCount: Integer;
1542    LSpan: Double;
1543  begin
        { lines 1544-1584 omitted: memo header and spike table }
1585      if LCount > 1 then
1586      begin
1587        LSpan := FAPG.SpikeTimes[LCount - 1] - FAPG.SpikeTimes[0];
1588        LLines.Add(Format('Mean ISI = %s ms  ->  mean frequency = %s Hz',
1589          [FloatText(LSpan / (LCount - 1), 3),
1590           FloatText(1000.0 * (LCount - 1) / LSpan, 2)]));
1591      end;
1592    end;
        { lines 1593-1604 omitted: paired-pulse note, EndUpdate }
1605  end;
```

e. Refractory period

The refractory period is measured with the paired-pulse protocol of Section 1.6. `PairedStimulus` builds a conditioning pulse J_1 of width w at $t_{on} = 1$ ms and a test pulse J_2 at $t_{on} + \Delta T$. `SecondSpikeOccurs` integrates the model and reports whether a spike crosses 0 mV after the onset of the test pulse. `SinglePulseThreshold` determines the rested threshold $J_{th,0}$ and `SecondPulseThreshold` the test threshold $J_{th}(\Delta T)$ by bisection on $[0, |J_2|_{max}]$. Bisection is valid because the response is monotone in the amplitude: if an amplitude elicits a spike, every larger one does as well. After 16 halvings the threshold is bracketed to $|J_2|_{max}/2^{16}$, about 0.008 µA/cm² for $|J_2|_{max} = 500$ µA/cm². The definitions used for the two phases of refractoriness are:

$$J_{th}(\Delta T) = \min\{|J_2|:\ \text{a second AP occurs}\}, \qquad \text{absolute: } J_{th}(\Delta T) > |J_2|_{max}, \qquad \text{relative: } J_{th,0} < J_{th}(\Delta T) \tag{2.3}$$

and the relative refractory period is taken to end when $J_{th}(\Delta T) \le 1.05\, J_{th,0}$. On the form, `btnRunRefClick` sweeps the interval, plots $J_{th}(\Delta T)$ with the rested threshold as a reference and the peak of the second action potential on a right axis, and derives both phase durations; `RunRefExample` simulates the paired-pulse response for the interval selected with the track bar.

```
613  function PairedStimulus(const J1, J2, Width, Interval: Double): TStimulus;
614  begin
615    Result := DefaultStimulus;
616    Result.Kind := skPairedPulse;
617    Result.JBase := 0.0;
618    Result.JLevel := J1;
619    Result.JLevel2 := J2;
620    Result.TOn := PAIRED_T_ON;
621    Result.PulseWidth := Width;
622    Result.Interval := Interval;
623  end;
625  function SecondSpikeOccurs(const P: THHParams; const J1, J2, Width,
626    Interval, Dt: Double; const Method: TIntegrator;
627    out SecondPeak: Double): Boolean;
628  var
629    Det: TSpikeDetector;
630    TTest: Double;
631    I: Integer;
```

```
632 begin
633   // The test response is a spike whose upward 0 mV crossing occurs after
634   // the onset of the second (test) pulse.
635   TTest := PAIRED_T_ON + Interval;
636   SimulateSpikesOnly(P, PairedStimulus(J1, J2, Width, Interval), Dt,
637     TTest + PAIRED_T_AFTER, Method, Det);
638   Result := False;
639   SecondPeak := NaN;
640   I := 0;
641   while (not Result) and (I < Det.Count) do
642   begin
643     if Det.CrossTimes[I] >= TTest then
644     begin
645       Result := True;
646       SecondPeak := Det.PeakValues[I];
647     end;
648     Inc(I);
649   end;
650 end;
652 function SinglePulseThreshold(const P: THHParams; const Sign, Width,
653   JMaxAbs, Dt: Double; const Method: TIntegrator): Double;
654 var
655   Lo, Hi, Mid: Double;
656   K: Integer;
657   Det: TSpikeDetector;
658   Stim: TStimulus;
659
660   function Fires(const A: Double): Boolean;
661   begin
662     Stim := PairedStimulus(Sign * A, 0.0, Width, 1.0E6);
663     SimulateSpikesOnly(P, Stim, Dt, PAIRED_T_ON + PAIRED_T_AFTER, Method, Det);
664     Result := Det.Count > 0;
665   end;
666
667 begin
668   // Smallest |J| of a single pulse that elicits an action potential
669   if not Fires(JMaxAbs) then
670     Exit(NaN);
671   Lo := 0.0;
672   Hi := JMaxAbs;
673   for K := 1 to BISECTION_STEPS do
674   begin
675     Mid := 0.5 * (Lo + Hi);
676     if Fires(Mid) then
677       Hi := Mid
678     else
679       Lo := Mid;
680   end;
681   Result := Hi;
682 end;
684 function SecondPulseThreshold(const P: THHParams; const J1, Width,
685   Interval, JMaxAbs, Dt: Double; const Method: TIntegrator): Double;
686 var
687   Lo, Hi, Mid, Pk, Sign: Double;
688   K: Integer;
689 begin
690   // Minimum |J2| of the test pulse that produces a second action potential.
691   // NaN means no response up to JMaxAbs: absolute refractory period.
692   if J1 < 0.0 then
693     Sign := -1.0
694   else
695     Sign := 1.0;
696   if not SecondSpikeOccurs(P, J1, Sign * JMaxAbs, Width, Interval, Dt,
697     Method, Pk) then
698     Exit(NaN);
699   Lo := 0.0;
700   Hi := JMaxAbs;
701   for K := 1 to BISECTION_STEPS do
702   begin
703     Mid := 0.5 * (Lo + Hi);
704     if SecondSpikeOccurs(P, J1, Sign * Mid, Width, Interval, Dt, Method, Pk) then
705       Hi := Mid
706     else
```

```
707       Lo := Mid;
708     end;
709     Result := Hi;
710   end;
1657  procedure TForm1.btnRunRefClick(Sender: TObject);
1658  var
1659    LParams: THHParams;
1660    LJ1, LWidth, LI0, LI1, LStep, LJMax, LDt, LPeak, LSign: Double;
1661    LCount, LIdx, LAbsIdx, LRelIdx: Integer;
1662    LLines: TStrings;
1663  begin
1664    if FBusy then
1665      Exit;
1666    FBusy := True;
1667    try
1668      LParams := ReadModelParams;
      { lines 1669-1675 omitted: read parameters from vleRef }
1676      if (LWidth <= 0.0) or (LStep <= 0.0) or (LI1 < LI0) or (LI0 < LWidth) then
1677        raise EConvertError.Create('Require width > 0, step > 0, ' +
1678          'interval start >= width and interval end >= interval start.');
1679      if LJ1 < 0.0 then
1680        LSign := -1.0
1681      else
1682        LSign := 1.0;
1683      LCount := Floor((LI1 - LI0) / LStep + 1.0E-9) + 1;
1684      SetLength(FRefInterval, LCount);
1685      SetLength(FRefThreshold, LCount);
1686      SetLength(FRefPeak, LCount);
1687
1688      Screen.Cursor := crHourGlass;
1689      try
1690        FRefBaseline := SinglePulseThreshold(LParams, LSign, LWidth, LJMax, LDt,
1691          SelectedMethod);
1692        for LIdx := 0 to LCount - 1 do
1693        begin
1694          FRefInterval[LIdx] := LI0 + LIdx * LStep;
1695          FRefThreshold[LIdx] := SecondPulseThreshold(LParams, LJ1, LWidth,
1696            FRefInterval[LIdx], LJMax, LDt, SelectedMethod);
1697          if SecondSpikeOccurs(LParams, LJ1, LJ1, LWidth, FRefInterval[LIdx],
1698            LDt, SelectedMethod, LPeak) then
1699            FRefPeak[LIdx] := LPeak
1700          else
1701            FRefPeak[LIdx] := NaN;
1702          SetStatus(Format('Refractory sweep %d / %d', [LIdx + 1, LCount]));
1703          Application.ProcessMessages;
1704        end;
1705      finally
1706        Screen.Cursor := crDefault;
1707      end;
1708
      { lines 1709-1726 omitted: plot serRefThr, serRefBase, serRefPeak }
1727
1728      // absolute refractory: no response up to JMax;
1729      // relative refractory ends when threshold <= 105 % of the rested value
1730      LAbsIdx := -1;
1731      for LIdx := 0 to LCount - 1 do
1732        if IsNan(FRefThreshold[LIdx]) then
1733          LAbsIdx := LIdx;
1734      LRelIdx := -1;
1735      LIdx := LAbsIdx + 1;
1736      while (LRelIdx < 0) and (LIdx < LCount) do
1737      begin
1738        if (not IsNan(FRefThreshold[LIdx])) and (not IsNan(FRefBaseline)) and
1739           (FRefThreshold[LIdx] <= 1.05 * FRefBaseline) then
1740          LRelIdx := LIdx;
1741        Inc(LIdx);
1742      end;
1743
      { lines 1744-1777 omitted: memo: refractory phases and table }
1778      SetStatus('Refractory sweep finished. Move the track bar to inspect ' +
1779        'single intervals.');
1780    except
1781      on E: Exception do
1782        ShowError(E.Message);
```

```
1783    end;
1784    FBusy := False;
1785    RunRefExample;
1786  end;
1788  procedure TForm1.RunRefExample;
      { lines 1789-1809 omitted: PlotTrace: Vm(t) into FRefExSeries[ASlot] }
1810      LSeries.Title := ATitle;
1811      LSeries.Active := True;
1812    end;
1813
1814  begin
1815    if FBusy then
1816      Exit;
1817    FBusy := True;
1818    try
1819      LParams := ReadModelParams;
      { lines 1820-1824 omitted: read parameters from vleRef }
1825      if (LWidth <= 0.0) or (LInterval < LWidth) then
1826        raise EConvertError.Create('Require width > 0 and interval >= width.');
1827      if LJ1 < 0.0 then
1828        LSign := -1.0
1829      else
1830        LSign := 1.0;
1831      LTEnd := 1.0 + LInterval + 20.0;
1832
1833      LStim := DefaultStimulus;
1834      LStim.Kind := skPairedPulse;
1835      LStim.TOn := 1.0;
1836      LStim.JLevel := LJ1;
1837      LStim.PulseWidth := LWidth;
1838      LStim.Interval := LInterval;
1839
1840      Screen.Cursor := crHourGlass;
1841      try
1842        LStim.JLevel2 := 0.0;
1843        SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
1844        PlotTrace(0, 'Conditioning pulse only');
1845
1846        LStim.JLevel2 := LJ1;
1847        SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
1848        PlotTrace(1, Format('Test pulse J2 = J1 = %s', [FloatText(LJ1, 1)]));
1849
1850        LThr := SecondPulseThreshold(LParams, LJ1, LWidth, LInterval, LJMax, LDt,
1851          SelectedMethod);
1852        if not IsNan(LThr) then
1853        begin
1854          LStim.JLevel2 := LSign * 1.05 * LThr;
1855          SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
1856          PlotTrace(2, Format('Test pulse J2 = 1.05 x threshold = %s',
1857            [FloatText(LStim.JLevel2, 1)]));
          { lines 1858-1871 omitted: status; absolute refractory hides series 3 }
1872      finally
1873        Screen.Cursor := crDefault;
1874      end;
1875      FinishChart(chtRefExample);
1876    except
1877      on E: Exception do
1878        ShowError(E.Message);
1879    end;
1880    FBusy := False;
1881  end;
```

f. How the Jin level programs the inter-pulse interval

`SteadyFiring` applies a constant J_{in} from $t = 0$, discards the spikes of the initial transient and computes the mean steady inter-spike interval from the peak times $t_{p,i}$, the firing frequency follows from the interval in milliseconds:

$$\mathrm{ISI}_i = t_{p,i} - t_{p,i-1}, \qquad \overline{\mathrm{ISI}} = \frac{1}{N_s} \sum_{t_{p,i-1} \geq t_{dis}} \mathrm{ISI}_i\,, \qquad f = \frac{1000}{\overline{\mathrm{ISI}}}\ \mathrm{Hz} \qquad (2.4)$$

The mechanism can be read from Equation (1.2). Between two spikes the membrane is charged by $J_{in} - J_{ion}$ through C_m, so the time needed to climb back to threshold scales roughly with $C_m \Delta V_{th} / |J_{in} - J_{ion}|$: a larger $|J_{in}|$ shortens the interval, while the refractoriness of Section 1.6 sets a lower bound below which no shortening is possible. `btnRunISIClick` sweeps J_{in}, plots $\overline{\mathrm{ISI}}$ and f against $|J_{in}|$ and draws three example spike trains.

```
714  procedure SteadyFiring(const P: THHParams; const J, Dt, TEnd,
715    TDiscard: Double; const Method: TIntegrator;
716    out NSpikes: Integer; out MeanISI: Double);
717  var
718    Stim: TStimulus;
719    Det: TSpikeDetector;
720    I, NUsed: Integer;
721    SumISI: Double;
722  begin
723    // Constant current J switched on at t = 0. The mean inter-spike interval is
724    // computed from spikes occurring after TDiscard (transient removed).
725    Stim := DefaultStimulus;
726    Stim.Kind := skStep;
727    Stim.JBase := 0.0;
728    Stim.JLevel := J;
729    Stim.TOn := 0.0;
730    Stim.TOff := TEnd + 1.0;
731    SimulateSpikesOnly(P, Stim, Dt, TEnd, Method, Det);
732    NSpikes := Det.Count;
733    SumISI := 0.0;
734    NUsed := 0;
735    for I := 1 to Det.Count - 1 do
736      if Det.PeakTimes[I - 1] >= TDiscard then
737      begin
738        SumISI := SumISI + (Det.PeakTimes[I] - Det.PeakTimes[I - 1]);
739        Inc(NUsed);
740      end;
741    if NUsed > 0 then
742      MeanISI := SumISI / NUsed
743    else
744      MeanISI := NaN;
745  end;
1908  procedure TForm1.btnRunISIClick(Sender: TObject);
1909  var
1910    LParams: THHParams;
1911    LJ0, LJ1, LJStep, LTRun, LTDis, LDt, LJ, LISI: Double;
1912    LCount, LIdx, LSpikes, LRheo, LRep, LBlock, LK: Integer;
1913    LStim: TStimulus;
1914    LResult: TAPGResult;
1915    LLines: TStrings;
1916    LTraceJ: array[0..2] of Double;
1917    LSeries: TFastLineSeries;
1918  begin
1919    if FBusy then
1920      Exit;
1921    FBusy := True;
1922    try
1923      LParams := ReadModelParams;
       { lines 1924-1929 omitted: read parameters from vleISI }
1930      if (LJStep <= 0.0) or (LTRun <= LTDis) or (LTDis < 0.0) then
1931        raise EConvertError.Create('Require step > 0 and 0 <= discard < run time.');
1932      LCount := Floor(Abs(LJ1 - LJ0) / LJStep + 1.0E-9) + 1;
1933      if LCount > 2000 then
1934        raise EConvertError.Create('Too many Jin levels (> 2000).');
1935      SetLength(FISIJin, LCount);
```

```
1936      SetLength(FISIValue, LCount);
1937      SetLength(FISICount, LCount);
1938
1939      Screen.Cursor := crHourGlass;
1940      try
1941        for LIdx := 0 to LCount - 1 do
1942        begin
1943          LJ := LJ0 + Sign(LJ1 - LJ0) * LIdx * LJStep;
1944          FISIJin[LIdx] := LJ;
1945          SteadyFiring(LParams, LJ, LDt, LTRun, LTDis, SelectedMethod, LSpikes,
1946            LISI);
1947          FISICount[LIdx] := LSpikes;
1948          FISIValue[LIdx] := LISI;
1949          if (LIdx mod 5) = 0 then
1950          begin
1951            SetStatus(Format('Jin sweep %d / %d', [LIdx + 1, LCount]));
1952            Application.ProcessMessages;
1953          end;
1954        end;
1955      finally
1956        Screen.Cursor := crDefault;
1957      end;
1958
          { lines 1959-1996 omitted: plot ISI, f and three example trains }
1997
1998      // summary: first AP, onset of repetitive firing, depolarization block
1999      LRheo := -1;
2000      LRep := -1;
2001      LBlock := -1;
2002      for LIdx := 0 to LCount - 1 do
2003      begin
2004        if (LRheo < 0) and (FISICount[LIdx] >= 1) then
2005          LRheo := LIdx;
2006        if (LRep < 0) and (not IsNan(FISIValue[LIdx])) then
2007          LRep := LIdx;
2008        if (LRep >= 0) and (LBlock < 0) and (LIdx > LRep) and
2009           IsNan(FISIValue[LIdx]) then
2010          LBlock := LIdx;
2011      end;
          { lines 2012-2040 omitted: memo: firing onset, block, table }
2041      SetStatus('Jin sweep finished.');
2042    except
2043      on E: Exception do
2044        ShowError(E.Message);
2045    end;
2046    FBusy := False;
2047  end;
```

2.2.3 Problem 2. Derivation of the APP Integration with Central and Forward Differences

Step 1, the partial differential equation. Consider the segment between x and $x + \Delta x$ of the transmission line in Figure 1.3. The axial current $i(x,t)$ flows through the internal and external resistances $r_i \Delta x$ and $r_o \Delta x$ in series, so Ohm's law gives the voltage drop along the segment, and dividing by Δx and letting $\Delta x \to 0$ gives the axial voltage gradient:

$$v(x+\Delta x,t) - v(x,t) = -\,i(x,t)(r_o + r_i)\Delta x \;\Rightarrow\; \frac{\partial v}{\partial x} = -(r_o + r_i)i \qquad (2.5)$$

At the node, the axial current that leaves the segment is smaller than the current that enters it by the membrane current $i_m \Delta x$ that crosses the membrane, which consists of a capacitive and an ionic part (Kirchhoff's current law):

$$i(x+\Delta x,t) - i(x,t) = -\,i_m \Delta x \;\Rightarrow\; \frac{\partial i}{\partial x} = -\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (2.6)$$

Differentiating Equation (2.5) with respect to x (with r_o and r_i uniform along the axon) and substituting Equation (2.6) eliminates the axial current:

$$\frac{\partial^2 v}{\partial x^2} = -(r_o + r_i)\frac{\partial i}{\partial x} = (r_o + r_i)\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (2.7)$$

Solving for the time derivative, adding the stimulus and using $i_{ion}/c_m = J_{ion}/C_m$ (both per-length quantities contain the factor $2\pi a$) gives the explicit form that is integrated by the program:

$$\frac{\partial v}{\partial t} = \frac{1}{(r_o + r_i)c_m}\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m} = D\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m} \qquad (2.8)$$

with $J_{ion} = J_{Na} + J_K + J_L$ for the active cable and $J_{ion} = g_m v$ for the passive (lecturer's RC) cable.

Step 2, central difference for the spatial derivative. On the grid $x_j = jh$, $t^n = nk$, write the Taylor series of v about x_j at a fixed time level n for a step $+h$ and a step $-h$:

$$v_{j+1}^n = v_j^n + h\,v_x + \frac{h^2}{2}v_{xx} + \frac{h^3}{6}v_{xxx} + \frac{h^4}{24}v_{xxxx} + O(h^5) \qquad (2.9)$$

$$v_{j-1}^n = v_j^n - h\,v_x + \frac{h^2}{2}v_{xx} - \frac{h^3}{6}v_{xxx} + \frac{h^4}{24}v_{xxxx} + O(h^5) \qquad (2.10)$$

Adding the two series cancels all odd derivatives:

$$v_{j+1}^n + v_{j-1}^n = 2v_j^n + h^2 v_{xx} + \frac{h^4}{12}v_{xxxx} + O(h^6) \qquad (2.11)$$

and solving for v_{xx} gives the central difference with a truncation error of second order in h:

$$\left.\frac{\partial^2 v}{\partial x^2}\right|_j^n = \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} - \frac{h^2}{12}v_{xxxx} \approx \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} \qquad (2.12)$$

Step 3, forward difference for the time derivative. The Taylor series in time at node j is

$$v_j^{n+1} = v_j^n + k\,v_t + \frac{k^2}{2}v_{tt} + O(k^3) \qquad (2.13)$$

so the forward difference is accurate to first order in k:

$$\left.\frac{\partial v}{\partial t}\right|_j^n = \frac{v_j^{n+1} - v_j^n}{k} - \frac{k}{2}v_{tt} \approx \frac{v_j^{n+1} - v_j^n}{k} \qquad (2.14)$$

Step 4, substitution and the explicit update. Substituting Equation (2.12) and (2.14) into Equation (2.8) at the point (x_j, t^n) and evaluating the membrane current at the known level n gives

$$\frac{v_j^{n+1} - v_j^n}{k} = D\,\frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} + \frac{J_{in,j}^n - J_{ion,j}^n}{C_m} \qquad (2.15)$$

and, multiplying by k, the explicit update used by the program:

$$v_j^{n+1} = v_j^n + r\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) + \frac{k}{C_m}\left(J_{in,j}^n - J_{ion,j}^n\right), \qquad r = \frac{Dk}{h^2} = \frac{k}{(r_o + r_i)c_m h^2} \tag{2.16}$$

For the passive cable of the lecture, $J_{ion,j}^n = g_m v_j^n$ and Equation (2.16) becomes

$$v_j^{n+1} = v_j^n + \frac{k}{h^2(r_o + r_i)c_m}\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) - \frac{k\, g_m}{C_m} v_j^n \tag{2.17}$$

in which the leakage term uses the voltage of the same node j. (The transcribed lecturer code uses `vm[0,i]`, the voltage of the first node, in this term this is most likely a transcription error and was not reproduced.) The local truncation error of the scheme is the sum of the two neglected terms, $\tau_j^n = -k/2\, v_{tt} - D\, h^2/12\, v_{xxxx} = O(k + h^2)$.

Step 5, gates of the active cable. In the active cable every node carries its own gates, which are advanced with the same forward difference applied to Equation (1.5), using the voltage of level n:

$$x_j^{n+1} = x_j^n + k\left[\alpha_x\left(v_j^n\right)\left(1 - x_j^n\right) - \beta_x\left(v_j^n\right)x_j^n\right], \qquad x \in \{m, h, n\} \tag{2.18}$$

Step 6, boundary conditions. The ends of the axon are sealed, which means that no axial current leaves the axon there. By Equation (2.5) this is $\partial v / \partial x = 0$. With a central difference across the boundary this condition introduces ghost nodes $v_{-1}^n = v_1^n$ and $v_N^n = v_{N-2}^n$, and substituting them into Equation (2.16) gives the end-node updates

$$v_0^{n+1} = v_0^n + 2r(v_1^n - v_0^n) + \frac{k}{C_m}(J_{in} - J_{ion})_0^n, \qquad v_{N-1}^{n+1} = v_{N-1}^n + 2r(v_{N-2}^n - v_{N-1}^n) + \frac{k}{C_m}(J_{in} - J_{ion})_{N-1}^n \tag{2.19}$$

Step 7, stability. Inserting a single Fourier mode $v_j^n = \xi^n e^{\mathrm{i}\beta jh}$ into the diffusion part of Equation (2.16) gives the amplification factor

$$\xi = 1 + r\left(e^{\mathrm{i}\beta h} - 2 + e^{-\mathrm{i}\beta h}\right) = 1 + r(2\cos\beta h - 2) = 1 - 4r\sin^2\frac{\beta h}{2} \tag{2.20}$$

Since $0 \le \sin^2(\beta h/2) \le 1$, ξ lies between $1 - 4r$ and 1, and the requirement $|\xi| \le 1$ for every wave number reduces to $1 - 4r \ge -1$:

$$r = \frac{Dk}{h^2} \le \frac{1}{2} \qquad \Leftrightarrow \qquad k \le k_{max} = \frac{h^2}{2D} \tag{2.21}$$

For the squid axon of Problem 3 ($a = 238$ µm, $R_i = 35.4$ Ω·cm, $r_o = 0$), $r_i = R_i/(\pi a^2) = 19{,}893$ Ω/cm and $c_m = 2\pi a C_m = 0.1495$ µF/cm. Because 1 Ω·µF = 1 µs, $D = 1/(r_i c_m) = 3.362 \times 10^{-4}$ cm²/µs $= 0.3362$ cm²/ms, and with $h = 1$ mm and $k = 0.01$ ms

the diffusion number is $r = 0.336 < 0.5$, the largest stable step at this h is $k_{max} = 0.0149$ ms. These numbers are reproduced exactly in the result memo of the program (Figure 2.7). The derivation of this section is also available in the program as sheet 5 ("Central/forward difference (Problem 2)"), opened from tab 5 or tab 6.

Relation to the lecture slide. The explicit update (2.16) with the passive membrane current is exactly the final expression of the lecture slide "Partial Differential Equation Solution", written there with $v_m(x \pm h, t)$ for $v_{j\pm1}^n$ and $v_m(x, t+k)$ for v_j^{n+1}: $v_m(x, t+k) = v_m(x,t) + \frac{k}{h^2}[v_m(x+h,t) - 2v_m(x,t) + v_m(x-h,t)][(r_o + r_i)C_m]^{-1} - k\, g_m v_m C_m^{-1}$, with C_m and g_m per unit length. The slide omits the brackets around the second difference, so that the factor $[(r_o + r_i)C_m]^{-1}$ seems to multiply only $v_m(x-h,t)$, but the derivation shows that it multiplies the whole difference, and the slide repeats the last line twice. Equation (2.17) is therefore the result asked for in Problem 2, and Equation (2.16) is its generalization to the active membrane.

Step 8, Crank-Nicolson extension. The lecturer's code and the senior's program also use a Crank-Nicolson form. Replacing the central difference in Equation (2.15) by the average of the levels n and $n+1$, while the membrane and stimulus terms stay at level n, gives

$$-\frac{r}{2}v_{j-1}^{n+1} + (1+r)v_j^{n+1} - \frac{r}{2}v_{j+1}^{n+1} = v_j^n + \frac{r}{2}\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) + \frac{k}{C_m}\left(J_{in,j}^n - J_{ion,j}^n\right) \qquad (2.21a)$$

which couples the three unknowns of each row and is solved for all nodes at once with the Thomas algorithm for tridiagonal systems (the sealed ends again double the off-diagonal coefficient of the first and last row). The same von Neumann analysis gives

$$\xi = \frac{1 - 2r\sin^2(\beta h/2)}{1 + 2r\sin^2(\beta h/2)}, \qquad |\xi| \le 1 \text{ for every } r > 0 \qquad (2.21b)$$

so the Crank-Nicolson scheme has no stability limit on k. (The transcribed lecturer code writes this average inside the explicit loop, using values of level $n+1$ that are not yet computed, the program solves the tridiagonal system instead.) When the axon is driven by the APG waveform, node 0 is not computed but set to $v_0^n = v_{APG}(t^n)$, which replaces the sealed-end condition at $x = 0$ in both schemes.

2.2.4 Problem 3. Program for Action Potential Propagation

`DefaultCableParams` sets the squid giant axon of Hodgkin and Huxley [1] ($a = 238$ µm, $R_i = 35.4$ Ω·cm, $r_o \approx 0$ for a large external bath), a 6 cm axon with $h = 1$ mm, $k = 0.01$ ms, a 12 ms run and one stimulus pulse of -150 µA/cm² for 0.5 ms applied to the first 2 mm, with the explicit scheme. `SimulateCable` is the implementation of Section 2.2.3. It validates the geometry and the excitation, computes r_i, c_m, D (with the factor 1000 that converts cm²/µs into cm²/ms), the diffusion number r and the passive length constant, and stops before integrating

if Equation (2.21) is violated by the explicit scheme. It then sets every node to the resting state and marches in time: for every node it forms the central difference of Equation (2.12), using the sealed-end form of Equation (2.19) at $j = 0$ and $j = N - 1$, it evaluates J_{ion} either from the HH currents with the node's own gates, which are advanced by Equation (2.18), or as $g_m v$ in the passive mode, and it writes the new voltage of Equation (2.16) into a second array, so that the whole new level is computed from level n only, as the explicit scheme requires. With the Crank-Nicolson option the same loop fills the right-hand side of Equation (2.21a) and `SolveTridiagonal` computes the new level. The excitation is either a train of current pulses (`PulseOn`) or the APG waveform imposed on node 0. After each step it records the peak V_m and the first upward 0 mV crossing of every node, interpolated linearly between the two time levels that bracket it, and stores a frame for plotting. Finally, the conduction velocity is computed from the crossing times at 25 % and 75 % of the axon length:

$$\theta = \frac{x_{75\%} - x_{25\%}}{t_c(x_{75\%}) - t_c(x_{25\%})} \cdot 10 \text{ m/s} \qquad (1 \text{ cm/ms} = 10 \text{ m/s}) \qquad (2.22)$$

Measuring between the 25 % and 75 % points keeps both the stimulated region and the sealed end, where the action potential is distorted, out of the velocity estimate.

```
761  function DefaultCableParams: TCableParams;
762  begin
763    Result.RadiusUm := 238.0;      // squid giant axon (Hodgkin & Huxley, 1952)
764    Result.RiOhmCm := 35.4;        // axoplasm resistivity (ohm.cm)
765    Result.RoOhmPerCm := 0.0;      // large external bath, ro << ri
766    Result.LengthCm := 6.0;
767    Result.DxCm := 0.1;
768    Result.Dt := 0.01;
769    Result.TEnd := 12.0;
770    Result.StimAmp := -150.0;
771    Result.StimDur := 0.5;
772    Result.StimLengthCm := 0.2;
773    Result.Mode := cmActiveHH;
774    Result.GmPassive := 0.677;
775    Result.SaveEvery := 2;
776    Result.Scheme := csExplicitFTCS;
777    Result.Drive := cdCurrentPulses;
778    Result.PulseCount := 1;
779    Result.PulseInterval := 15.0;
780    SetLength(Result.DriveV, 0);
781  end;
783  procedure SimulateCable(const HP: THHParams; const CP: TCableParams;
784    out R: TCableResult);
785  var
786    A, Phi, T, Lap, Jin, JNa, JK, JL, Jion, VmNow, InvH2, Src: Double;
787    V, VNew, Mg, Hg, Ng, VmPrev: TDoubleArray;
788    TriA, TriB, TriC, TriD, TriCp, TriDp: TDoubleArray;
789    Stimulated: array of Boolean;
790    J, K, NSteps, NFrames, Frame, I1, I2: Integer;
791    S: THHState;
792    Rt: THHRates;
793
794    function PulseOn(const Time: Double): Boolean;
795    var
796      P: Integer;
797    begin
798      // pulse p occupies [p * PulseInterval, p * PulseInterval + StimDur)
799      Result := False;
800      P := 0;
801      while (not Result) and (P < CP.PulseCount) do
802      begin
803        Result := (Time >= P * CP.PulseInterval) and
```

```
804           (Time < P * CP.PulseInterval + CP.StimDur);
805         Inc(P);
806       end;
807     end;
808
809     procedure SolveTridiagonal(const N: Integer);
810     var
811       Ii: Integer;
812       Den: Double;
813     begin
814       // Thomas algorithm for TriA[i] x[i-1] + TriB[i] x[i] + TriC[i] x[i+1]
815       // = TriD[i]; the matrix is diagonally dominant (1 + r > r), so no
816       // pivoting is needed. The solution is written into VNew.
817       TriCp[0] := TriC[0] / TriB[0];
818       TriDp[0] := TriD[0] / TriB[0];
819       for Ii := 1 to N - 1 do
820       begin
821         Den := TriB[Ii] - TriA[Ii] * TriCp[Ii - 1];
822         if Ii < N - 1 then
823           TriCp[Ii] := TriC[Ii] / Den
824         else
825           TriCp[Ii] := 0.0;
826         TriDp[Ii] := (TriD[Ii] - TriA[Ii] * TriDp[Ii - 1]) / Den;
827       end;
828       VNew[N - 1] := TriDp[N - 1];
829       for Ii := N - 2 downto 0 do
830         VNew[Ii] := TriDp[Ii] - TriCp[Ii] * VNew[Ii + 1];
831     end;
832
833     procedure SaveFrame(const Idx: Integer; const Time: Double);
834     var
835       Jj: Integer;
836     begin
837       R.T[Idx] := Time;
838       SetLength(R.Vm[Idx], R.NNodes);
839       for Jj := 0 to R.NNodes - 1 do
840         R.Vm[Idx][Jj] := AbsoluteVm(V[Jj]);
841     end;
842
843   begin
844     if (CP.DxCm <= 0.0) or (CP.LengthCm <= CP.DxCm) then
845       raise EArgumentException.Create('Require 0 < dx < axon length.');
846     if CP.RadiusUm <= 0.0 then
847       raise EArgumentException.Create('Axon radius must be positive.');
848     if CP.RiOhmCm + CP.RoOhmPerCm <= 0.0 then
849       raise EArgumentException.Create('ri + ro must be positive.');
850     CheckStepArguments(CP.Dt, CP.TEnd);
851     if (CP.Drive = cdCurrentPulses) and (CP.PulseCount < 1) then
852       raise EArgumentException.Create('The number of stimulus pulses must be at least 1.');
853     if (CP.Drive = cdCurrentPulses) and (CP.PulseCount > 1) and
854        (CP.PulseInterval <= CP.StimDur) then
855       raise EArgumentException.Create(
856         'The pulse interval must be longer than the pulse duration.');
857     if (CP.Drive = cdVoltageTrace) and
858        (Length(CP.DriveV) < Round(CP.TEnd / CP.Dt) + 1) then
859       raise EArgumentException.Create(
860         'The voltage trace at x = 0 is shorter than the simulated time.');
861
862     // cable constants
863     A := CP.RadiusUm * 1.0E-4;                        // cm
864     R.RiPerLen := CP.RiOhmCm / (Pi * A * A);          // ohm/cm
865     R.CmPerLen := HP.Cm * 2.0 * Pi * A;               // uF/cm
866     // ohm.uF = us, so 1/((ri+ro) cm) is in cm2/us; x1000 gives cm2/ms
867     R.D := 1000.0 / ((R.RiPerLen + CP.RoOhmPerCm) * R.CmPerLen);
868     R.R := R.D * CP.Dt / Sqr(CP.DxCm);
869     if CP.GmPassive > 0.0 then
870       // lambda^2 = rm/(ri+ro), rm = 1/(gm 2 pi a), gm in S/cm2
871       R.LambdaCm := Sqrt(1.0 / (CP.GmPassive * 1.0E-3 * 2.0 * Pi * A *
872         (R.RiPerLen + CP.RoOhmPerCm)))
873     else
874       R.LambdaCm := NaN;
875     R.VelocityMs := NaN;
876     R.NNodes := Round(CP.LengthCm / CP.DxCm) + 1;
877     // the explicit scheme needs r <= 1/2 (von Neumann); Crank-Nicolson is
```

```
878   // unconditionally stable for the diffusion term
879   R.Stable := (CP.Scheme = csCrankNicolson) or (R.R <= 0.5);
880   R.NSaved := 0;
881   SetLength(R.X, R.NNodes);
882   for J := 0 to R.NNodes - 1 do
883     R.X[J] := J * CP.DxCm;
884   if not R.Stable then
885     Exit;  // caller reports the violated condition k <= h^2/(2D)
886
887   // initial state: every node at rest
888   Phi := TemperatureFactor(HP);
889   S := SteadyState(0.0, Phi);
890   SetLength(V, R.NNodes);
891   SetLength(VNew, R.NNodes);
892   SetLength(Mg, R.NNodes);
893   SetLength(Hg, R.NNodes);
894   SetLength(Ng, R.NNodes);
895   SetLength(VmPrev, R.NNodes);
896   SetLength(Stimulated, R.NNodes);
897   SetLength(R.PeakVm, R.NNodes);
898   SetLength(TriA, R.NNodes);
899   SetLength(TriB, R.NNodes);
900   SetLength(TriC, R.NNodes);
901   SetLength(TriD, R.NNodes);
902   SetLength(TriCp, R.NNodes);
903   SetLength(TriDp, R.NNodes);
904   SetLength(R.TCross, R.NNodes);
905   for J := 0 to R.NNodes - 1 do
906   begin
907     V[J] := 0.0;
908     Mg[J] := S.M;
909     Hg[J] := S.H;
910     Ng[J] := S.N;
911     VmPrev[J] := AbsoluteVm(0.0);
912     R.PeakVm[J] := VmPrev[J];
913     R.TCross[J] := NaN;
914     Stimulated[J] := R.X[J] <= CP.StimLengthCm + 1.0E-9;
915   end;
916   if CP.Drive = cdVoltageTrace then
917   begin
918     V[0] := CP.DriveV[0];
919     VmPrev[0] := AbsoluteVm(V[0]);
920     R.PeakVm[0] := VmPrev[0];
921   end;
922
923   // Crank-Nicolson matrix (constant): (1 + r) on the diagonal, -r/2 beside
924   // it; the sealed ends use the ghost nodes v[-1] = v[1], v[N] = v[N-2],
925   // which doubles the off-diagonal coefficient of the first and last row
926   for J := 0 to R.NNodes - 1 do
927   begin
928     TriA[J] := -0.5 * R.R;
929     TriB[J] := 1.0 + R.R;
930     TriC[J] := -0.5 * R.R;
931   end;
932   TriA[0] := 0.0;
933   TriC[0] := -R.R;
934   TriA[R.NNodes - 1] := -R.R;
935   TriC[R.NNodes - 1] := 0.0;
936   if CP.Drive = cdVoltageTrace then
937   begin
938     // node 0 follows the imposed trace (Dirichlet row)
939     TriB[0] := 1.0;
940     TriC[0] := 0.0;
941   end;
942
943   NSteps := Round(CP.TEnd / CP.Dt);
944   if CP.SaveEvery < 1 then
945     NFrames := NSteps + 1
946   else
947     NFrames := NSteps div CP.SaveEvery + 1;
948   SetLength(R.T, NFrames);
949   SetLength(R.Vm, NFrames);
950   SaveFrame(0, 0.0);
951   Frame := 1;
```

```
952     InvH2 := 1.0 / Sqr(CP.DxCm);
953
954     // time marching (forward difference in time)
955     for K := 0 to NSteps - 1 do
956     begin
957       T := K * CP.Dt;
958       for J := 0 to R.NNodes - 1 do
959       begin
960         // central difference with sealed-end ghost nodes
961         if J = 0 then
962           Lap := (2.0 * V[1] - 2.0 * V[0]) * InvH2
963         else if J = R.NNodes - 1 then
964           Lap := (2.0 * V[J - 1] - 2.0 * V[J]) * InvH2
965         else
966           Lap := (V[J + 1] - 2.0 * V[J] + V[J - 1]) * InvH2;
967
968         if (CP.Drive = cdCurrentPulses) and Stimulated[J] and PulseOn(T) then
969           Jin := CP.StimAmp
970         else
971           Jin := 0.0;
972
973         if CP.Mode = cmActiveHH then
974         begin
975           S.V := V[J];
976           S.M := Mg[J];
977           S.H := Hg[J];
978           S.N := Ng[J];
979           IonicCurrents(HP, S, JNa, JK, JL);
980           Jion := JNa + JK + JL;
981           Rt := ComputeRates(V[J], Phi);
982           Mg[J] := Mg[J] + CP.Dt * (Rt.Am * (1.0 - Mg[J]) - Rt.Bm * Mg[J]);
983           Hg[J] := Hg[J] + CP.Dt * (Rt.Ah * (1.0 - Hg[J]) - Rt.Bh * Hg[J]);
984           Ng[J] := Ng[J] + CP.Dt * (Rt.An * (1.0 - Ng[J]) - Rt.Bn * Ng[J]);
985         end
986         else
987           Jion := CP.GmPassive * V[J];
988
989         // membrane and stimulus terms are always explicit (level n)
990         Src := (Jin - Jion) / HP.Cm;
991         if CP.Scheme = csCrankNicolson then
992           // right-hand side of the trapezoidal rule for the diffusion term
993           TriD[J] := V[J] + CP.Dt * (0.5 * R.D * Lap + Src)
994         else
995           VNew[J] := V[J] + CP.Dt * (R.D * Lap + Src);
996       end;
997
998       if CP.Scheme = csCrankNicolson then
999       begin
1000        if CP.Drive = cdVoltageTrace then
1001          TriD[0] := CP.DriveV[K + 1];
1002        SolveTridiagonal(R.NNodes);
1003      end
1004      else if CP.Drive = cdVoltageTrace then
1005        VNew[0] := CP.DriveV[K + 1];
1006
1007      for J := 0 to R.NNodes - 1 do
1008      begin
1009        V[J] := VNew[J];
1010        VmNow := AbsoluteVm(V[J]);
1011        if VmNow > R.PeakVm[J] then
1012          R.PeakVm[J] := VmNow;
1013        if IsNan(R.TCross[J]) and (VmPrev[J] < SPIKE_LEVEL) and
1014           (VmNow >= SPIKE_LEVEL) then
1015          // linear interpolation of the crossing instant
1016          R.TCross[J] := T + CP.Dt * (SPIKE_LEVEL - VmPrev[J]) /
1017            (VmNow - VmPrev[J]);
1018        VmPrev[J] := VmNow;
1019      end;
1020
1021      if (CP.SaveEvery >= 1) and (((K + 1) mod CP.SaveEvery) = 0) and
1022         (Frame < NFrames) then
1023      begin
1024        SaveFrame(Frame, (K + 1) * CP.Dt);
1025        Inc(Frame);
```

```
1026      end
1027      else if (CP.SaveEvery < 1) and (Frame < NFrames) then
1028      begin
1029        SaveFrame(Frame, (K + 1) * CP.Dt);
1030        Inc(Frame);
1031      end;
1032    end;
1033    R.NSaved := Frame;
1034
1035    // conduction velocity between 25 % and 75 % of the axon length
1036    I1 := Round(0.25 * (R.NNodes - 1));
1037    I2 := Round(0.75 * (R.NNodes - 1));
1038    if (I2 > I1) and (not IsNan(R.TCross[I1])) and (not IsNan(R.TCross[I2])) and
1039       (R.TCross[I2] > R.TCross[I1]) then
1040      // 1 cm/ms = 10 m/s
1041      R.VelocityMs := 10.0 * (R.X[I2] - R.X[I1]) / (R.TCross[I2] - R.TCross[I1]);
1042  end;
```

The model exhibits propagation because of the coupling term $D\ \partial^2 v/\partial x^2$ together with the regenerative membrane current. When node j fires, its depolarization makes the central difference at node $j+1$ negative (in the lecturer's sign), so Equation (2.16) drives v_{j+1} toward depolarization. As soon as the neighbour reaches threshold, its own sodium current takes over and produces a full action potential, which in turn depolarizes node $j+2$. The wave therefore travels with constant amplitude, set by the membrane, and at a constant velocity, set by how fast the axial current can charge the next segment, that is by D and the membrane kinetics. In the passive mode the same coupling only spreads and attenuates the stimulated depolarization with the length constant λ, and no action potential appears away from the stimulation site. Comparing both modes in the same program is the direct demonstration asked for in Problem 3.

On the form, `FillAPPEditor` fills the parameter table with lengths in millimetres, and the two preset buttons select either the squid axon (`btnPresetSquidClick`: active cable, one current pulse) or the lecturer's setting (`btnPresetLecturerClick`, values from `LecturerCableParams`), which is also the default when the program starts. The lecturer's simulation uses a passive 2 mm cable with $dx = 0.4$ mm and $dt = 10$ µs that is driven at $x = 0$ by the action potentials of the APG model, so the preset sets the passive mode, the APG waveform as excitation, a 60 ms run and recording sites at 0, 0.4, …, 2.0 mm. A radius of 5 µm gives a passive length constant of about 1 mm and an attenuation over 2 mm of the size seen in the lecturer's result. A squid axon conducting at about 12 m/s with an action potential lasting about 2 ms produces a wave about 2.4 cm long, so a 2 mm squid axon would be shorter than the action potential itself, which is why the squid preset uses a 6 cm axon. `btnRunAPPClick` reads the parameters, converts millimetres to centimetres, builds the APG waveform with the tab-1 protocol (`BuildAPGStimulus` and `SimulateAPG` with the cable time step) when it is selected as excitation, calls `SimulateCable`, reports a violated stability condition together with k_{max}, and then calls `PlotCable` (Vm(t) at evenly spaced recording sites and Vm(x) at evenly spaced instants), `ReportCable` (excitation, scheme, cable constants, r, λ, θ and a table of peak voltages and crossing times) and `UpdateAPPFrame`, which moves the time cursor, the bold Vm(x) profile and the overlay of the space-time view when the time slider or the Play button changes the frame. The space-time view offers a heat map, a 3D waterfall and a 3D mesh of

$V_m(x,t)$. `OpenSheet` and `ShowEquationSheet` open the derivation windows described in Section 2.2.1.

```
933  function TForm1.LecturerCableParams: TCableParams;
934  begin
935    // 2 mm cable with dx = 0.4 mm and dt = 10 us as in the lecturer's APP
936    // simulation. A radius of 5 um gives a passive length constant of about
937    // 1 mm, so the attenuation over 2 mm is of the size shown in the
938    // lecturer's result; the HH membrane is unchanged (ISS-023, ISS-029).
939    Result := DefaultCableParams;
940    Result.RadiusUm := 5.0;
941    Result.LengthCm := 0.2;          // 2 mm
942    Result.DxCm := 0.04;             // h = 0.4 mm
943    Result.Dt := 0.01;               // k = 10 us
944    Result.TEnd := 60.0;
945    Result.StimLengthCm := 0.04;     // used by the pulse drive only
946    Result.PulseCount := 4;
947    Result.PulseInterval := 15.0;
948  end;
950  procedure TForm1.ApplyAPPPreset(const ACable: TCableParams; AMode, ADrive,
951    AScheme: Integer);
952  begin
953    StopPlay;
954    FillAPPEditor(ACable, C_SITES_DEFAULT);
955    rgCableMode.ItemIndex := AMode;
956    rgAPPDrive.ItemIndex := ADrive;
957    rgAPPScheme.ItemIndex := AScheme;
958  end;
960  procedure TForm1.FillAPPEditor(const ACable: TCableParams; ASites: Integer);
961  begin
962    // lengths are shown in mm; RoundTo removes binary noise of cm * 10
963    SetupEditor(vleAPP);
       { lines 964-973 omitted: rows a, Ri, ro, length, h, k, T, Jin, duration }
974    // gm of the passive cable = chord conductance of the resting HH membrane
975    AddParam(vleAPP, K_CGM, RoundTo(RestingConductance(DefaultHHParams), -4));
976    AddParam(vleAPP, K_CSITES, ASites);
977    AddParam(vleAPP, K_CNP, ACable.PulseCount);
978    AddParam(vleAPP, K_CPI, ACable.PulseInterval);
979  end;
981  procedure TForm1.btnPresetSquidClick(Sender: TObject);
982  begin
983    ApplyAPPPreset(DefaultCableParams, C_MODE_ACTIVE, C_DRIVE_PULSES,
984      C_SCHEME_FTCS);
985    SetStatus('Preset: squid giant axon, active cable, one current pulse ' +
986      '(a = 238 um, 60 mm axon, h = 1 mm). Press Run APP simulation.');
987  end;
989  procedure TForm1.btnPresetLecturerClick(Sender: TObject);
990  begin
991    ApplyAPPPreset(LecturerCableParams, C_MODE_PASSIVE, C_DRIVE_APG,
992      C_SCHEME_FTCS);
993    SetStatus('Preset: lecturer scale, passive cable driven by the APG ' +
994      'waveform of tab 1 (2 mm, dx = 0.4 mm, dt = 10 us, 60 ms). Press Run ' +
995      'APP simulation.');
996  end;
2078  procedure TForm1.btnRunAPPClick(Sender: TObject);
2079  var
2080    LParams: THHParams;
2081    LCable: TCableParams;
2082    LSites: Integer;
2083    LStim: TStimulus;
2084    LDrive: TAPGResult;
2085  begin
2086    StopPlay;
2087    try
2088      LParams := ReadModelParams;
2089      LCable := DefaultCableParams;
       { lines 2090-2100 omitted: read cable parameters (mm to cm) }
2101      LSites := EnsureRange(ParamInt(vleAPP, K_CSITES), 2, C_MAX_SITES);
2102      LCable.PulseCount := ParamInt(vleAPP, K_CNP);
2103      LCable.PulseInterval := ParamValue(vleAPP, K_CPI);
2104      if rgCableMode.ItemIndex = C_MODE_PASSIVE then
```

```
2105        LCable.Mode := cmPassive
2106      else
2107        LCable.Mode := cmActiveHH;
2108      if rgAPPScheme.ItemIndex = C_SCHEME_CN then
2109        LCable.Scheme := csCrankNicolson
2110      else
2111        LCable.Scheme := csExplicitFTCS;
2112      if rgAPPDrive.ItemIndex = C_DRIVE_APG then
2113      begin
2114        // APG waveform of the tab-1 protocol, computed with the cable time
2115        // step so that sample n belongs to t = n k
2116        LCable.Drive := cdVoltageTrace;
2117        BuildAPGStimulus(LStim);
2118        SimulateAPG(LParams, LStim, LCable.Dt, LCable.TEnd, SelectedMethod, LDrive);
2119        LCable.DriveV := Copy(LDrive.V);
2120      end
2121      else
2122        LCable.Drive := cdCurrentPulses;
2123      // keep roughly 600 stored frames for plotting and animation
2124      LCable.SaveEvery := Max(1, Round(LCable.TEnd / LCable.Dt) div 600);
2125
2126      Screen.Cursor := crHourGlass;
2127      try
2128        SimulateCable(LParams, LCable, FCable);
2129      finally
2130        Screen.Cursor := crDefault;
2131      end;
2132      Inc(FCableRunId);   // invalidates the cached 3D surface
2133
2134      if not FCable.Stable then
2135      begin
2136        FCableValid := False;
2137        RefreshSpaceTime;
2138        ShowError(Format('Explicit scheme unstable: r = k/((ro+ri) cm h^2) = %s ' +
2139          '> 0.5. Use k <= %s ms or a larger h (mm).', [FloatText(FCable.R, 3),
2140          FloatText(0.5 * Sqr(LCable.DxCm) / FCable.D, 5)]));
2141        Exit;
2142      end;
2143      FCableValid := True;
2144      FAPPFrame := 0;
2145      PlotCable(LSites);
2146      BuildHeatMap;
2147      ReportCable(LCable);
2148      tbAPPTime.Max := Max(1, FCable.NSaved - 1);
2149      tbAPPTime.Position := 0;
2150      UpdateAPPFrame;
2151      SetStatus(Format('APP finished: %d nodes, %d frames. Press Play or move ' +
2152        'the time slider.', [FCable.NNodes, FCable.NSaved]));
2153    except
2154      on E: Exception do
2155        ShowError(E.Message);
2156    end;
2157  end;
2159  procedure TForm1.PlotCable(ASites: Integer);
2160  var
2161    LSeries: TFastLineSeries;
2162    LIdx, LK, LNode, LFrame: Integer;
2163  begin
2164    ClearChart(chtAPPTime);
2165    ClearChart(chtAPPSpace);
2166    // a coarse grid cannot have more distinct recording sites than nodes;
2167    // the form holds C_MAX_SITES site and snapshot series, unused ones hide
2168    ASites := Min(ASites, FCable.NNodes);
       { lines 2169-2175 omitted: show only the first ASites series }
2176    // Vm(t) at evenly spaced recording sites, coloured by position
2177    for LIdx := 0 to ASites - 1 do
2178    begin
2179      LNode := Round(LIdx * (FCable.NNodes - 1) / (ASites - 1));
2180      LSeries := FSiteSeries[LIdx];
2181      LSeries.Title := Format('x = %s mm',
2182        [FloatText(FCable.X[LNode] * C_MM_PER_CM, 2)]);
2183      LSeries.SeriesColor := OrderedLineColor(LIdx, ASites);
2184      for LK := 0 to FCable.NSaved - 1 do
2185        LSeries.AddXY(FCable.T[LK], FCable.Vm[LK][LNode]);
```

```
2186    end;
2187    // serTimeCursor (vertical line, not in the legend) is set in UpdateAPPFrame
2188    FinishChart(chtAPPTime);
2189
2190    // Vm(x) at evenly spaced instants (thin, coloured by time) + live profile
2191    for LIdx := 1 to ASites do
2192    begin
2193      LFrame := Round(LIdx * (FCable.NSaved - 1) / (ASites + 1));
2194      LSeries := FSnapSeries[LIdx - 1];
2195      LSeries.Title := Format('t = %s ms', [FloatText(FCable.T[LFrame], 2)]);
2196      LSeries.SeriesColor := OrderedLineColor(LIdx - 1, ASites);
2197      for LK := 0 to FCable.NNodes - 1 do
2198        LSeries.AddXY(FCable.X[LK] * C_MM_PER_CM, FCable.Vm[LFrame][LK]);
2199    end;
2200    FinishChart(chtAPPSpace);
2201  end;
2203  procedure TForm1.ReportCable(const AParams: TCableParams);
2204  var
2205    LLines: TStrings;
2206    LIdx, LNode, LPrev: Integer;
2207  begin
2208    LLines := memAPP.Lines;
2209    LLines.BeginUpdate;
2210    try
        { lines 2211-2240 omitted: memo: mode, scheme, D, r, lambda, velocity }
2241      LLines.Add('   x(mm)   peak Vm(mV)   t(Vm=0 mV)(ms)');
2242      { Up to 11 evenly spaced nodes; a short cable (for example the 6 nodes
2243        of the lecturer preset) has fewer, so a node already listed is skipped }
2244      LPrev := -1;
2245      for LIdx := 0 to 10 do
2246      begin
2247        LNode := Round(LIdx * (FCable.NNodes - 1) / 10);
2248        if LNode = LPrev then
2249          Continue;
2250        LPrev := LNode;
2251        LLines.Add(Format('%7s   %10s   %12s',
2252          [FloatText(FCable.X[LNode] * C_MM_PER_CM, 2),
2253          FloatText(FCable.PeakVm[LNode], 2), FloatText(FCable.TCross[LNode], 3)]));
2254      end;
2255    finally
2256      LLines.EndUpdate;
2257    end;
2258  end;
2260  procedure TForm1.UpdateAPPFrame;
2261  var
2262    LIdx: Integer;
2263    LTime: Double;
2264  begin
2265    if not FCableValid then
2266      Exit;
2267    FAPPFrame := EnsureRange(tbAPPTime.Position, 0, FCable.NSaved - 1);
2268    LTime := FCable.T[FAPPFrame];
2269    lblAPPTime.Caption := Format('t = %s ms', [FloatText(LTime, 2)]);
2270    serLiveProfile.BeginUpdate;
2271    try
2272      serLiveProfile.Clear;
2273      for LIdx := 0 to FCable.NNodes - 1 do
2274        serLiveProfile.AddXY(FCable.X[LIdx] * C_MM_PER_CM,
2275          FCable.Vm[FAPPFrame][LIdx]);
2276      serLiveProfile.Title := Format('time slider: t = %s ms',
2277        [FloatText(LTime, 2)]);
2278    finally
2279      serLiveProfile.EndUpdate;
2280    end;
2281    serTimeCursor.Clear;
2282    serTimeCursor.AddXY(LTime, C_VM_LO - 10.0);
2283    serTimeCursor.AddXY(LTime, C_VM_HI + 10.0);
2284    RefreshSpaceTime;
2285  end;
3015  procedure TForm1.OpenSheet(AIndex: Integer);
3016  begin
3017    if (AIndex < SHEET_FIRST) or (AIndex > SHEET_LAST) then
3018      Exit;
```

```
  try
    ShowEquationSheet(Self, AIndex);
    SetStatus(Format('Sheet %d opened: %s', [AIndex, SheetTitle(AIndex)]));
  except
    on E: Exception do
    begin
      AppendLog('ERROR', 'Sheet ' + IntToStr(AIndex) + ': ' + E.Message);
      ShowError('The sheet could not be opened: ' + E.Message);
    end;
  end;
end;
```

```
        procedure ShowEquationSheet(AOwnerForm: TForm; AIndex: Integer);
                                                                    var
                                                  LForm: TSheetForm;
                                                                  begin
      if (AIndex < SHEET_FIRST) or (AIndex > SHEET_LAST) then
      raise EArgumentOutOfRangeException.CreateFmt('Sheet %d does not exist.',
                                                          [AIndex]);
                  if GSheets[AIndex] <> nil then
                                                                  begin
    // already open: restore it if minimised and bring it to the front
                    if GSheets[AIndex].WindowState = wsMinimized then
                                  GSheets[AIndex].WindowState := wsNormal;
                                                GSheets[AIndex].Show;
                                          GSheets[AIndex].BringToFront;
                                                                  Exit;
                                                                  end;
      LForm := TSheetForm.CreateSheet(AOwnerForm, AIndex, OpenSheetCount);
                                  GSheets[AIndex] := LForm;
                                                LForm.Show;
end;
```

2.2.5 Problem 4. The Hodgkin-Huxley Model in Two Research Works

Paper 1: a model of the pacemaking neuron of the respiratory central pattern generator

Amini, Bidani, Zwischenberger and Clark [7] address a central question of respiratory neurophysiology: how much of the breathing rhythm is generated by the intrinsic membrane properties of individual neurons and how much by network connectivity. The respiratory central pattern generator is a population of medullary neurons that drives the phrenic motor neurons and hence the diaphragm. Its putative pacemaker, the pre-inspiratory neuron of the pre-Bötzinger complex (PBC), fires before inspiration and sets the respiratory rate. The authors build an HH-type model of this neuron as the driving element of a larger network model and use it to dissect the ionic mechanism of bursting and the phase sensitivity of the rhythm.

The first important aspect is that the paper keeps the complete Hodgkin-Huxley formalism and only enlarges the set of conductances. Every voltage-gated current is written as a maximal conductance multiplied by powers of gating variables and a driving force, for example $I_{CaT} = g_{CaT} d_T^2 f_T (v - 37.0)$, and every gating variable follows first-order kinetics with $x_\infty = \alpha/(\alpha + \beta)$ and $\tau_x = c/(\alpha + \beta)$ [7, Table II]. Several rate functions have exactly the form $a(v + b)/\left(e^{c(v+b)} - 1\right)$ of α_m and α_n in the original model [1] and therefore inherit the same removable singularity that is handled analytically by `ExpRatio` in this assignment. Instead of the three currents of the squid axon, the PBC neuron contains fast and persistent

sodium currents (I_{Na}, I_{NaP}), T-, N- and L-type calcium currents, delayed-rectifier and transient potassium currents (I_K, I_A, I_D), a calcium-activated potassium current (I_{KCa}), a hyperpolarization-activated cation current (I_h), pump and exchanger currents and an intracellular calcium balance [7, Fig. 2]. The second aspect is an explicit temperature correction: because voltage-clamp data of the different currents were recorded at different temperatures, time constants and maximal conductances were scaled by $Q_{10}^{(T_1-T_2)/10}$ to 302 K with current-specific Q_{10} values between 1.28 and 5.1 [7, Table I]. This is the same mechanism as the factor $\phi = 3^{(T-6.3)/10}$ of Hodgkin and Huxley [1], which the simulator of this assignment exposes as its temperature parameter. The third aspect is numerical: the stiff system was integrated with the variable-step CVODE solver rather than with a fixed-step Runge-Kutta scheme.

The model reproduces the interburst trajectory, the rising baseline during a burst, the burst duration and period and the peak intraburst frequency of recorded PBC neurons [7, Fig. 2(a)]. Its main value lies in in-silico channel-block experiments. Removing I_{NaP} abolishes bursting and leaves tonic spiking at about 15 Hz, so the persistent sodium current sets the depolarized baseline on which bursts ride. I_{Na} generates the spikes and recruits I_{CaL} and I_{CaN}, the resulting rise of intracellular calcium activates I_{KCa}, which terminates the burst [7, Fig. 3]. Phasic current pulses applied at different phases of the cycle give a phase response curve: stimulation during the burst produces a phase delay that decreases and reverses into an advance, while stimulation during the repolarizing phase gives a plateau of constant phase advance [7, Fig. 4]. Two inconsistencies should be noted when citing the paper. The Methods section specifies a 150 nA stimulus, whereas the caption of Fig. 4 states 100 nA, and the phase change is defined as $\Delta P = P_0 - P_1$ with positive values said to indicate a delay, although a delay lengthens the perturbed period ($P_1 > P_0$) and gives a negative ΔP, as the Results text itself states. The authors also note that several potassium kinetics were adapted from neighbouring respiratory neurons because direct voltage-clamp data were scarce.

Paper 2: high-speed, low-resource digital implementation of the HH model using base-2 functions

Haghiri, Naderi, Ghanbari and Ahmadi [8] treat the HH model as the most biologically faithful but also the most expensive neuron model for neuromorphic hardware: the powers m^3 and n^4, divisions and exponential rate functions require multipliers, dividers and look-up resources that limit how many neurons fit on a chip. The paper keeps the four-state structure of the model and replaces every costly nonlinear term with a short linear combination of base-2 exponentials, which digital logic evaluates with shifters, adders and subtractors only. The mathematical core is a projection onto an orthonormal basis obtained by Gram-Schmidt orthonormalization of $\{2^{-2v}, 2^{-v}, 1, 2^{v}, 2^{2v}\}$ on $[0,1]$, each term is approximated by its first five generalized Fourier coefficients after the voltage is normalized to the unit interval [8]. A detail directly relevant to this assignment is the sign convention: Haghiri et al. use the modern form with $V_{Na} = 115$ mV, $V_K = -12$ mV and $\alpha_m = (2.5 - 0.1V)/(e^{2.5-0.1V} - 1)$, which is

exactly the lecturer's form with $v = -V$. The two descriptions are therefore the same model, and the alignment $V_m = -(v + 70)$ of Problem 1 equals $V_m = V - 70$ in their notation.

The approximation is evaluated at three levels. Time-domain traces for stimuli of 2 to 80 µA/cm² agree with the original model with a mean correlation of 90.4 %, mean RMSE of 7.4 mV and mean NRMSE of 9.34 % [8, Table II]. Dynamically, the reduced system keeps the type and nearly the location of its fixed points, which the authors show with nullclines, Jacobian matrices and phase portraits. In hardware the model is discretized with the forward Euler method ($\Delta t = 2^{-7}$), pipelined with 30-bit fixed-point words and implemented on a Virtex-4 FPGA using 4 % of the slice flip-flops, 19 % of the LUTs, no hardware multipliers and a maximum clock of 200 MHz [8, Table III]. Some points require a critical reading. The error evaluation is reported for "dt = 10 ms", which cannot integrate HH kinetics and is presumably a lost unit prefix, the same defect as the "10 sec" step of the lecture slide (Section 1.3). The single action potential shown for 2 µA/cm² is most likely an initial-condition artefact, because the reference traces start with the gates near zero, with the simulator of this assignment, zero initial gates produce one spike at 4.1 ms for $J_{in} = -2$ µA/cm² and even at 5.4 ms without any input, whereas a membrane started at rest does not fire at this level. The paper also does not explain how the standard HH equations, which do not burst intrinsically, produce the bursting patterns of its Fig. 9, and its introduction dates the Nobel Prize of Hodgkin and Huxley to 1961 instead of 1963.

Table 2.1. *Comparison of the HH model in this assignment and in the two reviewed papers*

Aspect	**This assignment**	**Amini et al. [7]**	**Haghiri et al. [8]**
Purpose	Teaching APG, refractoriness, propagation	Mechanism of the respiratory rhythm	Low-cost neuromorphic hardware
Currents	I_{Na}, I_K, I_L (squid axon)	More than a dozen, incl. I_{NaP}, I_{Ca}, I_{KCa}, I_h	I_{Na}, I_K, I_L (squid axon)
Sign convention	HH 1952, $V_m = -(v + 70)$	Absolute physiological mV	Modern, $V = -v$
Temperature	$\phi = 3^{(T-6.3)/10}$	Current-specific Q_{10}, 302 K	Not used
Integration	RK4, $\Delta t = 0.01$ ms, FTCS cable	CVODE, variable step	Forward Euler, fixed point
Nonlinear terms	Exact, singularities handled	Exact (HH form)	Base-2 approximations
Validation	Python cross-check, HH 1952 velocity	Recorded PBC activity	RMSE, phase plane, FPGA traces

Impact of the Hodgkin-Huxley work on medical and biological technology

The deepest impact of Hodgkin and Huxley [1] is methodological. They showed that excitability can be explained quantitatively by a small set of voltage-dependent conductances described by gating variables with first-order kinetics, and that this description, measured under voltage clamp, predicts the action potential, the refractory period and the conduction velocity. The two papers show how far this template travels. In Amini et al. [7] it becomes a tool of respiratory physiology: more than a dozen conductances in exactly the HH formalism reproduce the bursting of the respiratory pacemaker, pharmacological block becomes a computational experiment that identifies the currents that start and stop a breath, and the phase response curve indicates when an electrical stimulus can advance or delay the next breath, which is the knowledge needed for drug targeting and for electrical control of breathing. The same conductance-based approach was carried into cardiac electrophysiology by Noble's modification of the HH equations for Purkinje fibre and pacemaker potentials [9], the starting point of present-day models of drug action on cardiac ion channels. Haghiri et al. [8] illustrate the engineering line of impact that because the HH model remains the reference for biological fidelity, neuromorphic engineers try to keep its dynamics while reducing its cost, and a multiplier-free HH neuron running at 200 MHz on a small FPGA makes real-time networks of biophysically realistic neurons feasible. The authors position this for the study of disorders

such as epilepsy and Parkinson's disease and for bio-hybrid systems, citing neuroprostheses that restore communication between neuronal populations [10] and closed-loop neuroprostheses [11], which require artificial neurons that respond to living tissue on its own millisecond time scale. Together, the papers show the HH model both as a scientific instrument and as an engineering specification. The simulator developed in Problems 1 and 3 sits at the origin of both lines: it solves the same equations, shows the same refractoriness and input-dependent firing rate, and reproduces the propagation velocity reported in 1952.

2.3 Execution Analysis

2.3.1 Action Potential Generation and Output Alignment

Figure 2.3 shows tab 1 for the default step protocol, $J_{in} = -10$ µA/cm² from 1 ms to 61 ms, integrated with the coupled RK4 at $\Delta t = 0.01$ ms. Before the step the membrane stays exactly at $V_m = -70$ mV: the exported data of the program give $J_{Na} = 1.220$, $J_K = -4.400$ and $J_L = 3.184$ µA/cm² at rest, whose sum is zero to within 4×10^{-3} µA/cm², with $m_\infty = 0.0529$, $h_\infty = 0.5961$ and $n_\infty = 0.3177$, which confirms that the initial condition is a true equilibrium. After the onset the membrane fires repetitively. Each action potential rises from about -60 mV to a peak of about $+35$ mV, falls below rest to an after-hyperpolarization of about -80 mV, close to the potassium reversal potential of -82 mV, and then depolarizes slowly toward the next threshold. Thanks to the alignment of Equation (2.2) the trace has the shape and the voltage range of a physiological recording, whereas the raw variable v of the model would show downward spikes from 0 to about -105 mV. The current chart shows the inward sodium current as a short, large pulse at the upstroke followed by the slower outward potassium current, and the gate and conductance charts show m rising almost instantly from 0.05 to nearly 1 while h falls and n rises with a delay, exactly the separation of time scales described in Section 1.2.

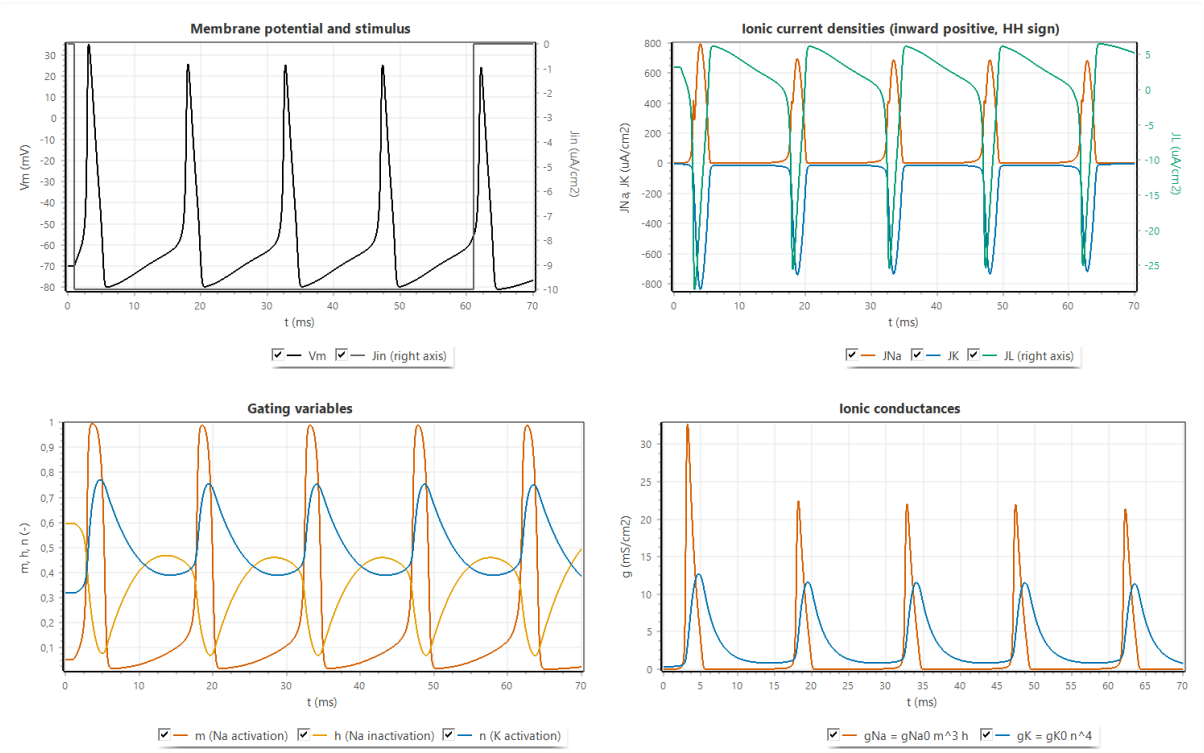


Figure 2.3. *Tab 1 for a step of $J_{in} = -10$ μA/cm²: membrane voltage and stimulus, ionic currents (HH sign, inward positive), gating variables and conductances*

The response to different constant levels is summarized in Table 2.2, obtained from 100 ms runs with the step starting at 1 ms. Levels of -2 μA/cm² or weaker remain subthreshold, -3 to -5 μA/cm² give a single action potential followed by a stable depolarized state, and repetitive firing starts between -6 and -7 μA/cm². Stronger inputs shorten the interval and raise the first peak slightly, until at -100 μA/cm² only one spike occurs and the membrane remains depolarized (depolarization block). The lecturer's staircase protocol produces action potentials at 3.23, 28.36 and 40.01 ms, one at the onset of each new level: the -5 μA/cm² level only elicits a single onset spike (Table 2.2), and the -10 and -20 μA/cm² levels last only 13 and 12 ms, which is not longer than the interval needed for a second spike at these levels. Selecting the lecturer's sequential RK4 changes the spike times by at most 0.01 ms (for example an interval of 17.15 ms instead of 17.14 ms at -7 μA/cm²), so the output of the program agrees with the lecturer's method within the resolution of the plots, and the senior's program, which uses the same equations with a small constant added to the rate denominators, shows qualitatively the same waveforms.

Table 2.2. *Response of the space-clamped membrane to constant input levels (coupled RK4, $\Delta t = 0.01$ ms, step from 1 ms, 100 ms run)*

J_{in} (µA/cm²)	Action potentials	Last ISI (ms)	First peak V_m (mV)
-2	0	-	-
-3	1	-	32.51
-5	1	-	34.05
-7	6	17.14	34.69
-10	7	14.64	35.26
-20	9	11.56	36.30
-40	11	9.21	37.49
-100	1, then block	-	40.00

2.3.2 Gating Phase Plots

Figure 2.4 shows the trajectories of the three gates against V_m for the same run, drawn over the steady-state curves. The trajectory of m stays close to $m_\infty(V_m)$ during the slow phases and separates from it only during the fast upstroke and downstroke, because τ_m is shorter than 0.5 ms, the corresponding loop is narrow. The trajectories of h and n form wide loops that lie far from their steady-state curves, because τ_h and τ_n are several milliseconds: during the spike h and n cannot follow the voltage, and after it they recover only gradually. The first loop starts from the resting values ($h = 0.60$, $n = 0.32$) and is different from the following loops, which start from $h \approx 0.45$ because recovery between spikes is incomplete under the sustained current, this incomplete recovery is the refractoriness that limits the firing rate. Because the dots are plotted at a constant time step, their spacing shows the speed of the state: the dots are dense during the slow interspike depolarization and sparse on the upstroke, where V_m changes by more than 100 mV within about a millisecond.

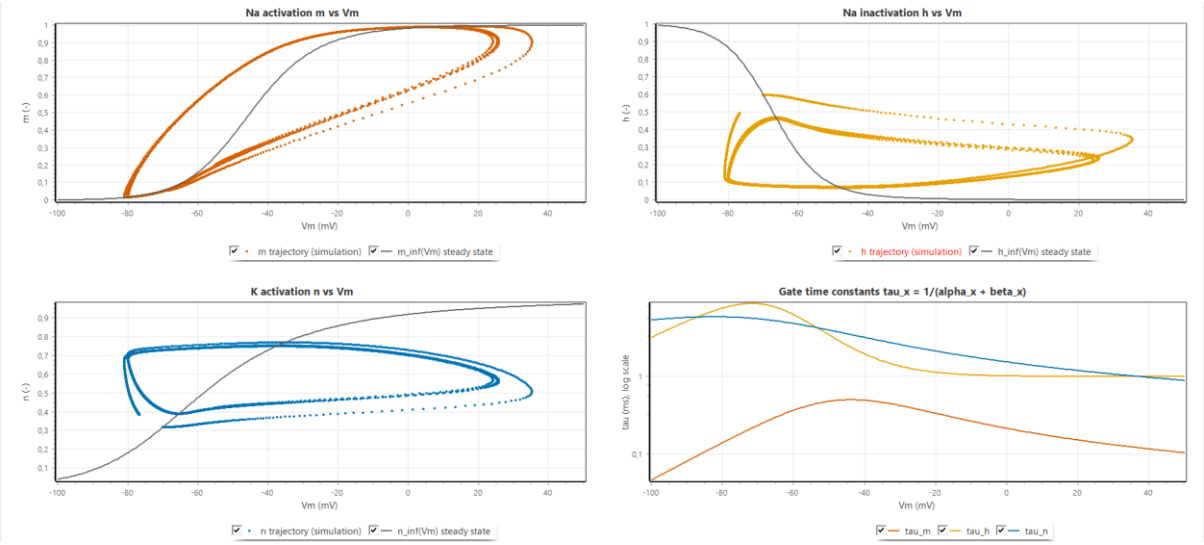


Figure 2.4. *Tab 2: trajectories of* m, h *and* n *against* V_m *(dots at a constant time step) over the steady states* $x_\infty(V_m)$*, and the time constants* $\tau_x(V_m)$

2.3.3 Refractory Period

Figure 2.5 shows the refractory sweep with a conditioning pulse $J_1 = -40$ µA/cm² of 0.5 ms and the test interval varied in steps of 0.5 ms, selected values are listed in Table 2.3. The rested single-pulse threshold is 13.27 µA/cm². For intervals up to 3.5 ms no test pulse up to 500 µA/cm² produces a second action potential, which is the absolute refractory period. At 4.5 ms the threshold is just below the search limit (496 µA/cm²), and it then falls steeply, to 85.8 µA/cm² at 7.5 ms and 37.3 µA/cm² at 10.5 ms, which is the relative refractory period. The threshold returns to within 5 % of the rested value at about 15 ms (13.75 µA/cm² at 15.5 ms). Between about 16.5 and 25 ms the threshold is even lower than at rest, with a minimum of 11.20 µA/cm² at 18.5 ms, about 16 % below the rested value. This supernormal phase is consistent with the gate dynamics, by then n has decayed close to its resting value, while h, which recovered quickly during the after-hyperpolarization, is still slightly above its resting value, so that more sodium conductance is available than at rest. The right axis shows that a second action potential elicited late in the relative refractory period is smaller (30.3 mV at 10.5 ms against 36.3 mV at 17.5 ms), because the available sodium conductance is still reduced. The lower chart shows the corresponding paired-pulse responses for the interval chosen with the track bar.

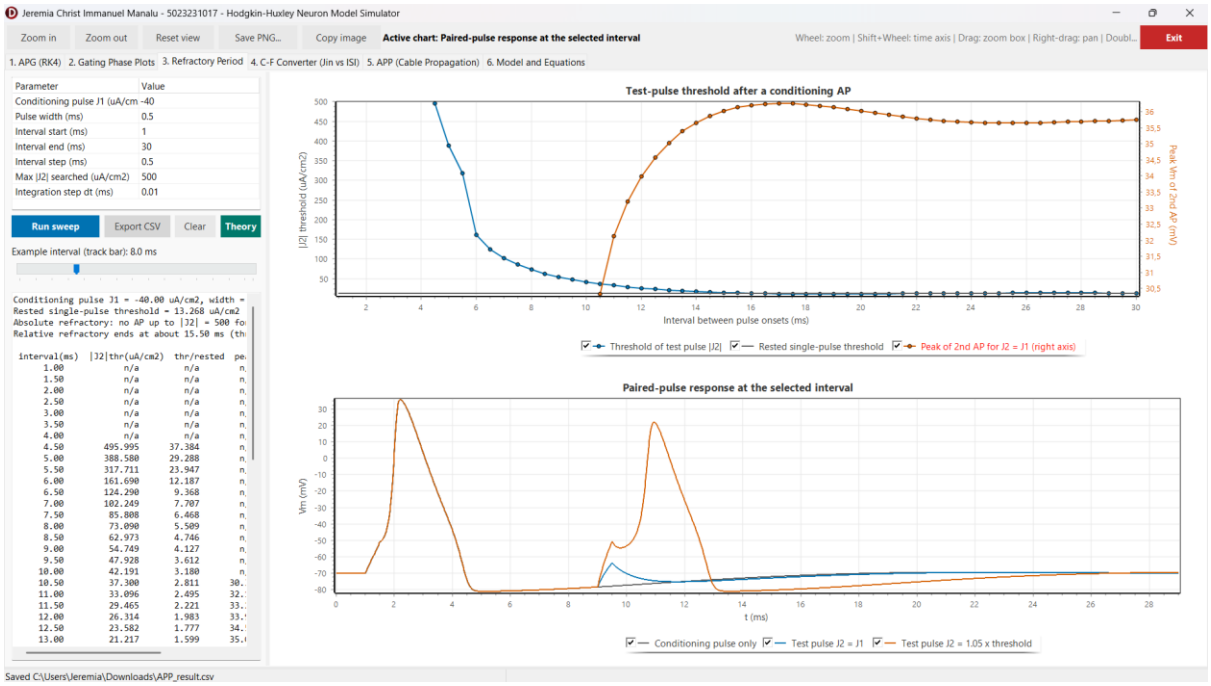


Figure 2.5. *Tab 3: test-pulse threshold and peak of the second action potential against the interval between pulse onsets, and the paired-pulse response at the selected interval*

Table 2.3. *Test-pulse threshold after a conditioning action potential ($J_1 = -40$ μA/cm², width 0.5 ms, rested threshold 13.27 μA/cm²)*

Interval (ms)	Test threshold $\|J_2\|_{th}$ (μA/cm²)	Ratio to rested threshold
3.5	none up to 500	-
4.5	496.0	37.4
6.5	124.3	9.37
8.5	63.0	4.75
10.5	37.3	2.81
12.5	23.6	1.78
15.5	13.75	1.04
18.5	11.20	0.84
24.5	13.42	1.01
29.5	13.52	1.02

2.3.4 How the Jin Level Programs the Inter-Pulse Interval

Figure 2.6 and Table 2.4 show the steady firing obtained in tab 4 for constant levels from 0 to −68 μA/cm². Up to −2 μA/cm² the membrane is silent, from −3 to −6 μA/cm² it fires once or twice and then settles, and from −7 μA/cm² it fires tonically with an interval of 17.15 ms

(58.3 Hz). The interval then decreases monotonically with $|J_{in}|$, steeply at first and more slowly at large inputs: 14.64 ms (68.3 Hz) at -10, 11.57 ms (86.5 Hz) at -20, 9.21 ms (108.6 Hz) at -40 and 8.33 ms (120.1 Hz) at -54 µA/cm². At -55 µA/cm² and beyond, firing stops after two or three spikes because the membrane stays depolarized and sodium inactivation does not recover (depolarization block). The input level therefore programs the inter-pulse interval within a range of roughly 8 to 17 ms. The saturation at large inputs reflects the refractory period of Section 2.3.3: however fast the membrane is recharged, a new spike cannot occur before h and n have partly recovered. The example traces in the lower chart illustrate the shorter intervals and the slightly smaller spike amplitude at strong inputs.

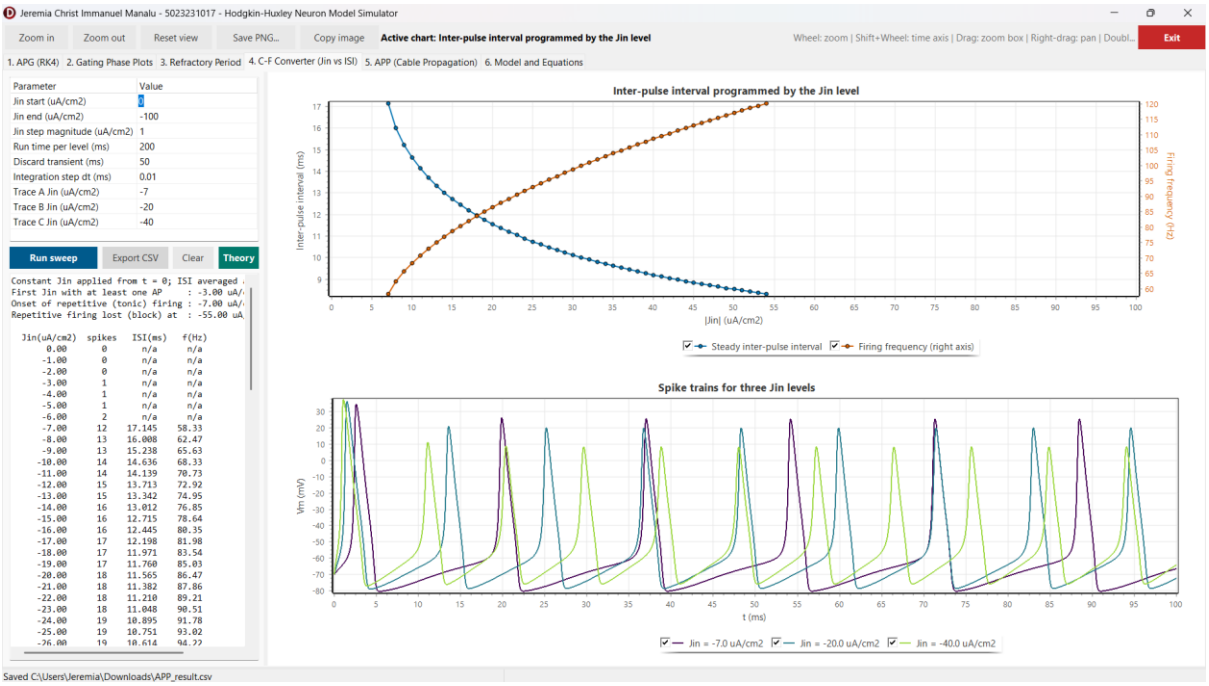


Figure 2.6. *Tab 4: steady inter-pulse interval and firing frequency against $|J_{in}|$, and spike trains for three input levels*

Table 2.4. *Steady inter-pulse interval and firing frequency for selected input levels (program export)*

J_{in} (µA/cm²)	Mean ISI (ms)	Frequency (Hz)
−6	- (2 spikes, no steady firing)	-
−7	17.15	58.3
−10	14.64	68.3
−15	12.72	78.6
−20	11.57	86.5
−30	10.13	98.7
−40	9.21	108.6
−54	8.33	120.1
−55	depolarization block	-

2.3.5 Action Potential Propagation

Figure 2.7 shows tab 5 for the active squid-axon cable. The result memo confirms the cable constants derived in Section 2.2.3 ($r_i = 19{,}893$ Ω/cm, $c_m = 0.14954$ µF/cm, $D = 0.33616$ cm²/ms, $r = 0.3362$). The Vm(t) chart shows the same action potential arriving at successive recording sites with a constant delay, and the Vm(x) chart shows the wave as a pulse that moves along the axon with an unchanged shape. The 0 mV crossing times grow linearly with distance, from 0.652 ms at the stimulated end through 1.486 ms at 12 mm and 2.980 ms at 30 mm to 4.476 ms at 48 mm, which gives a conduction velocity of 12.04 m/s at 6.3 °C. The peak voltage is 33.03 mV along the whole middle part of the axon. It is slightly higher at the stimulated end (34.07 mV), where the stimulus adds to the membrane current, and at the sealed far end (37.32 mV), where no axial current can flow onward and the charge accumulates in the last segments. The spatial extent of the action potential, about 2 to 3 cm, is consistent with the product of the velocity and the spike duration.

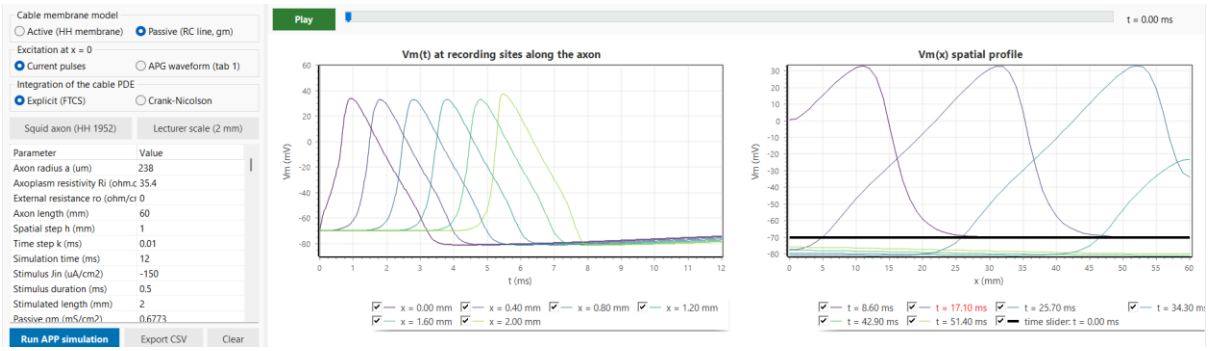


Figure 2.7. *Tab 5, active cable with the squid-axon preset: parameters, Vm(t) at six recording sites and Vm(x) at six instants*

Table 2.5 summarizes the parameter variations. Halving the spatial step to 0.5 mm (with a smaller time step to keep $r \leq 0.5$) changes the velocity by less than 2 %, which agrees with the second-order spatial accuracy of the scheme. Raising the temperature to 18.5 °C increases the velocity to 18.2 m/s, against 18.8 m/s calculated and 21.2 m/s measured by Hodgkin and Huxley [1], the agreement with their calculation is within about 3 %. In the passive mode no action potential appears: the depolarization decays from -35.2 mV at the stimulated end to -68.1 mV at 15 mm, in line with the computed length constant of 7.0 mm, which demonstrates that propagation requires the regenerative sodium current. A thin active fibre of 1 µm radius on a 2 mm axon propagates at about 0.78 m/s, consistent with $\theta \propto \sqrt{a}$. A step larger than k_{max} is rejected by the program with the message predicted by Equation (2.21). Figure 2.8 shows the space-time behaviour in the 3D views, where the propagating action potential appears as a straight ridge in the (x, t) plane whose slope is the inverse of the velocity, and Figure 2.9 shows the derivation window of the cable equation opened from the same tab.

Table 2.5. *Propagation results for parameter variations*

Case	**Result**
Active cable, squid axon, 6.3 °C, $h = 1$ mm, $k = 0.01$ ms	$\theta = 12.04$ m/s, peak 33.03 mV, $r = 0.336$
Active cable, $h = 0.5$ mm	$\theta = 12.25$ m/s
Active cable, 18.5 °C	$\theta = 18.2$ m/s (HH 1952: 18.8 calculated, 21.2 measured)
Passive cable, $g_m = 0.6773$ mS/cm²	no AP, -35.2 mV at 0 mm to -68.1 mV at 15 mm, $\lambda = 7.0$ mm
Thin active fibre, $a = 1$ µm, 2 mm, $h = 0.1$ mm	$\theta \approx 0.78$ m/s, $r = 0.14$

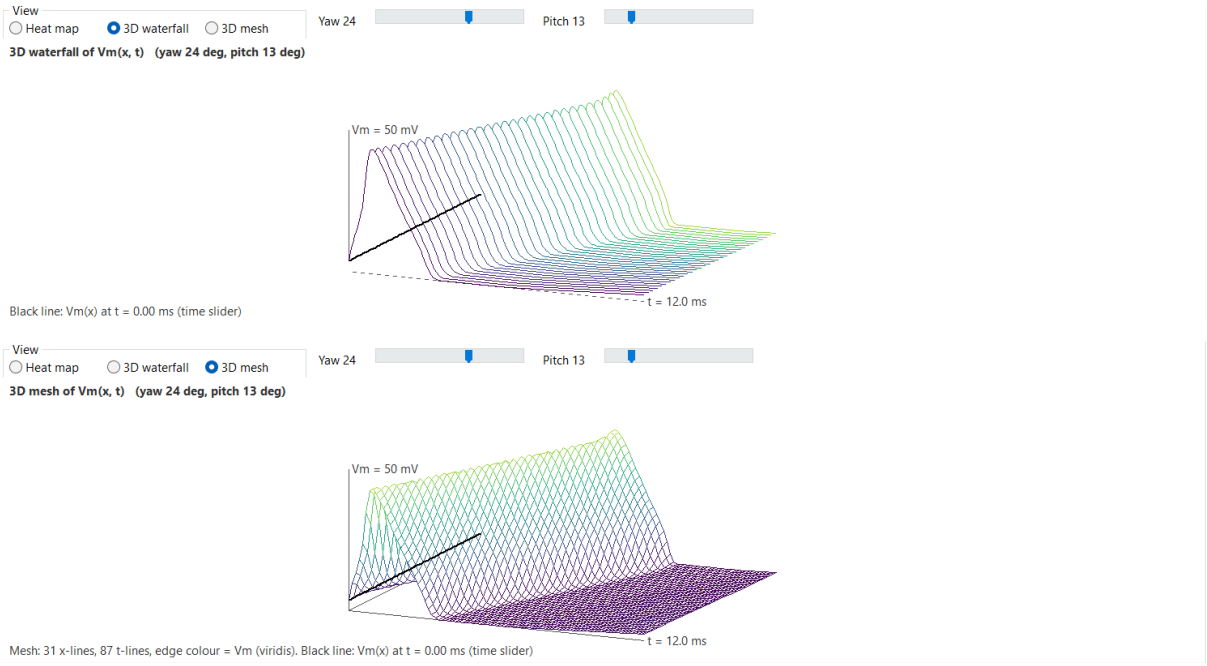


Figure 2.8. *Space-time views of $V_m(x,t)$ for the active cable: (a) 3D waterfall, (b) 3D mesh, the bold line is the spatial profile at the time-slider position*

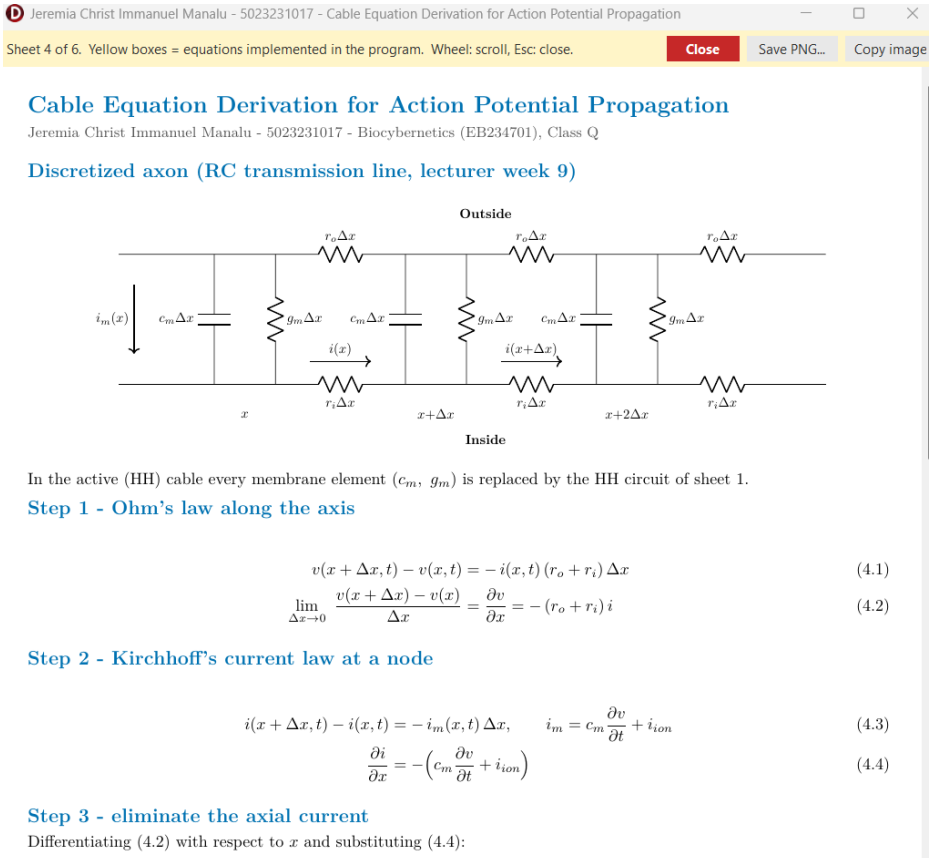


Figure 2.9. *Derivation window of the cable equation (sheet 4) opened from the main form, the yellow boxes are the equations used by the cable solver*

Compared with the reference programs, the results differ in two respects that are intended. The senior's propagation program updates the voltage with a pure diffusion term and an arbitrarily chosen coefficient applied to a stored APG trace, without any membrane current, so its signal can only spread and decay, the active cable of this work regenerates the action potential at every node and therefore propagates without decrement, which is what Problem 3 asks the model to exhibit. The lecturer's program, in contrast, solves the passive cable equation driven by the APG output, which the lecturer-scale preset reproduces (Figure 2.10), while the squid-axon preset was added because its velocity can be checked against the 1952 paper.

Figure 2.10 shows the lecturer-scale preset. The APG model, driven by $J_{in} = -10$ µA/cm² from 1 ms, fires at 2.91, 17.84, 32.49 and 47.12 ms, and these four action potentials are imposed at $x = 0$. Along the passive cable each of them appears as a separate wave that becomes smaller and broader with distance: the peaks at 0, 0.4, 0.8, 1.2, 1.6 and 2.0 mm are 35.3, −9.5, −32.7, −45.0, −50.4 and −51.9 mV, with a passive length constant of 1.02 mm. The repeated ridges in the 3D views are therefore successive action potentials in time on one axon, not several axons, which is the same picture as in the lecturer's and the senior's plots. The Crank-Nicolson scheme gives the same peaks within 0.1 mV at this grid ($r = 0.044$). Its advantage appears at larger diffusion numbers: for the squid axon with $h = 0.5$ mm and $k = 0.01$ ms ($r = 1.345$) the explicit scheme is refused, whereas the Crank-Nicolson scheme runs and gives 12.12 m/s, within 1 % of the explicit result with a four times smaller step (12.25 m/s). With the active membrane and the same APG drive on a 0.1 mm grid ($r = 0.71$, Crank-Nicolson), every wave is regenerated and travels at 1.95 m/s. A train of three current pulses 15 ms apart likewise produces three propagating action potentials at 12.04 m/s on the squid axon.

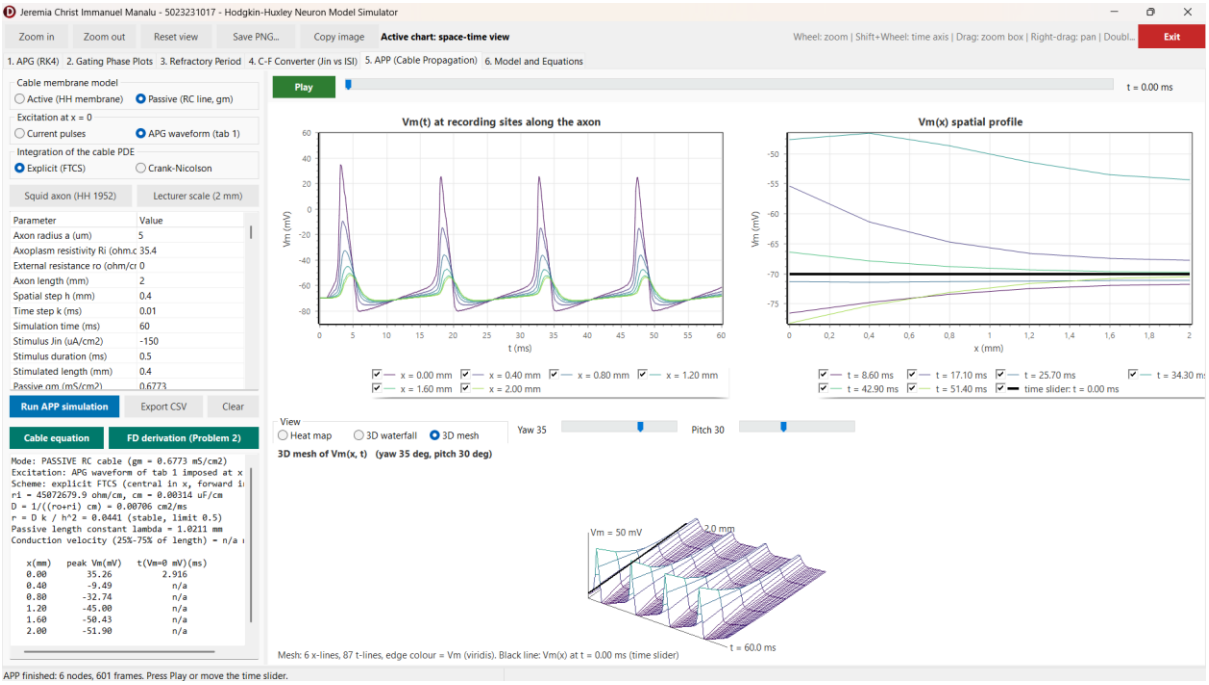


Figure 2.10. *Tab 5, lecturer-scale preset: passive 2 mm cable driven at $x = 0$ by the APG waveform of tab 1 ($J_{in} = -10$ µA/cm²), Vm(t) at 0 to 2.0 mm and the space-time view of the four attenuated waves.*

2.4 Evaluation and Discussion

The results agree with the theoretical baseline of Chapter I in every aspect examined. The voltage equation (1.2) with the currents (1.3) and the kinetics (1.5) to (1.9) reproduces the resting equilibrium exactly and generates action potentials of the expected amplitude and duration once the output is aligned with Equation (2.2). The phase plots make the separation of time scales of Figure 1.2 directly visible: the fast gate m tracks its steady state, while the slow gates h and n lag behind and recover only gradually, and this lag is the common cause of the two behaviours of Problem 1. It produces the absolute refractory period of about 4 ms and the relative refractory period ending near 15 ms, and it bounds the inter-pulse interval from below, so that the input level can program the interval only between about 17 and 8 ms before depolarization block sets in. The supernormal phase found after the relative refractory period is not an artefact of the threshold search, which is monotone and resolved to 0.008 µA/cm², but a property of the HH kinetics that follows from the different recovery rates of h and n.

For propagation, the derivation of Problem 2 and the program of Problem 3 are consistent, where the cable constants and the stability number printed by the program equal the hand-calculated values, the velocity converges with second-order behaviour when h is halved, and the comparison of the active and passive modes isolates the regenerative sodium current as the reason why an action potential propagates without decrement while a passive signal decays within a few length constants. The velocity of 18.2 m/s at 18.5 °C is within about 3 % of the value calculated by Hodgkin and Huxley [1], the remaining difference to their measurement of 21.2 m/s is of the same size as the difference between their own calculation and measurement, and it is expected because only the rate constants are temperature-scaled and the axon is modelled as a uniform cylinder.

Several limitations must be kept in mind. The explicit FTCS scheme is only first-order accurate in time and is stable only for $r \leq 1/2$, so fine spatial grids require small time steps, the Crank-Nicolson option removes this restriction for the diffusion term, while the membrane term remains explicit. The sealed-end boundary raises the peak at the far end, and spikes are defined by a 0 mV crossing, so graded responses below 0 mV are not counted. Three discrepancies in the lecture material were identified and handled explicitly: the time step printed as “10 sec” (10 µs), the term $0.1(v + 2.5)$ printed in α_m instead of $0.1(v + 25)$, and the node index of the leakage term in the transcribed propagation code. In addition, the final line of the slide lacks the brackets around the second difference. In each case the program follows the derivation and the original paper, and offers the lecturer’s integration method for direct comparison.

CHAPTER III. CONCLUSION

A Hodgkin-Huxley neuron simulator was developed in Delphi 13 with a numerical core that is independent of the user interface. The action potential is generated by integrating the four HH equations with the classical RK4 method and, for comparison, with the lecturer's sequential RK4 at a step of 0.01 ms, and the output is aligned to the membrane voltage with $V_m = -(v + 70)$ mV, which turns the downward spikes of the 1952 sign convention into physiological action potentials from -70 mV to about $+35$ mV. A paired-pulse threshold search shows an absolute refractory period of about 4 ms and a relative refractory period ending near 15 ms, and a constant input programs the inter-pulse interval from about 17 ms (58 Hz) at -7 μA/cm² to about 8 ms (120 Hz) at -54 μA/cm², beyond which depolarization block occurs.

The integration of the cable equation was derived with a central difference in space and a forward difference in time, giving the explicit update $v_j^{n+1} = v_j^n + r\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) + k(J_{in} - J_{ion})/C_m$ with $r = k/[(r_o + r_i)c_m h^2]$ and the stability condition $r \leq 1/2$, which for the passive membrane is the final expression of the lecture slide, and a Crank-Nicolson variant removes the stability limit. Implemented with an HH membrane at every node, the scheme shows a regenerating action potential that travels at constant amplitude and at 12.0 m/s for the squid axon at 6.3 °C (18.2 m/s at 18.5 °C, close to the 18.8 m/s calculated by Hodgkin and Huxley), whereas the passive cable driven by the APG waveform, as in the lecturer's program, carries each action potential as a wave that decays within about 1 mm. Propagation is therefore exhibited by the combination of axial current flow and the regenerative sodium current.

The two reviewed papers show the reach of the Hodgkin-Huxley formalism: extended with more than a dozen conductances, it explains the bursting of the respiratory pacemaker, and approximated with base-2 functions, it runs in real time on small FPGAs for neuromorphic and neuroprosthetic applications. All equations used by the program, including the derivation of Problem 2, are available in the program as separate sheets and are highlighted in this report.

REFERENCES

[1] A. L. Hodgkin and A. F. Huxley, “A quantitative description of membrane current and its application to conduction and excitation in nerve,” *J. Physiol.*, vol. 117, no. 4, pp. 500-544, Aug. 1952, doi: 10.1113/jphysiol.1952.sp004764.

[2] A. Arifin, “Hodgkin-Huxley neuron model,” lecture notes, Biocybernetics (EB234701), Dept. Biomed. Eng., Inst. Teknologi Sepuluh Nopember, Surabaya, Indonesia, 2026.

[3] A. Arifin, “Hodgkin-Huxley neuron model simulation,” lecture slides and source code, Biocybernetics (EB234701), Dept. Biomed. Eng., Inst. Teknologi Sepuluh Nopember, Surabaya, Indonesia, 2026.

[4] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[5] J. Malmivuo and R. Plonsey, *Bioelectromagnetism: Principles and Applications of Bioelectric and Biomagnetic Fields*. New York, NY, USA: Oxford Univ. Press, 1995.

[6] K. W. Morton and D. F. Mayers, *Numerical Solution of Partial Differential Equations: An Introduction*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2005.

[7] B. Amini, A. Bidani, J. B. Zwischenberger, and J. W. Clark, Jr., “A model of the pacemaking neuron of the respiratory central pattern generator,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 13, no. 2, pp. 120-124, Jun. 2005, doi: 10.1109/TNSRE.2005.847379.

[8] S. Haghiri, A. Naderi, B. Ghanbari, and A. Ahmadi, “High speed and low digital resources implementation of Hodgkin-Huxley neuronal model using base-2 functions,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 275-287, Jan. 2021, doi: 10.1109/TCSI.2020.3026076.

[9] D. Noble, “A modification of the Hodgkin-Huxley equations applicable to Purkinje fibre action and pace-maker potentials,” *J. Physiol.*, vol. 160, no. 2, pp. 317-352, Feb. 1962, doi: 10.1113/jphysiol.1962.sp006849.

[10] S. Buccelli *et al.*, “A neuromorphic prosthesis to restore communication in neuronal networks,” *iScience*, vol. 19, pp. 402-414, Sep. 2019, doi: 10.1016/j.isci.2019.07.046.

[11] T. Levi, P. Bonifazi, P. Massobrio, and M. Chiappalone, “Editorial: Closed-loop systems for next-generation neuroprostheses,” *Front. Neurosci.*, vol. 12, Art. no. 26, Feb. 2018, doi: 10.3389/fnins.2018.00026.